

Table des matières

Introduction générale	1
Chapitre 1 : Étude préalable et analyse des besoins	2
1.1 Introduction	2
1.2 Contexte et définition de la mission	2
1.3 Problématique	2
1.4 Objectifs à atteindre	2
1.5 Critères d'acceptation du démonstrateur	3
1.6 Analyse et critique de l'existant	4
1.7 Solution proposée	4
1.8 Méthodologie et organisation du travail	4
1.9 Planning de référence sur six semaines	4
1.10 Présentation du projet	5
1.11 Acteurs et responsabilités	5
1.12 Besoins fonctionnels	5
1.13 Besoins de sécurité	6
1.14 Traçabilité et sécurité des données	6
1.15 Authentification et contrôle d'accès	6
1.16 TLS et réseau privé virtuel	6
1.17 Conteneurisation et orchestration	7
1.18 Conclusion du chapitre	7
Chapitre 2 : Conception de l'architecture proposée	8
2.1 Cas d'utilisation et responsabilités	8
2.2 Architecture générale	8
2.3 Architecture réseau de l'hébergement OVH	9
2.4 Architecture Kubernetes et stockage	10
2.5 Modèle de données	10
2.6 Cycle de vie d'un bagage	11
2.7 Séparation des protections	12
2.8 Organisation du code et des responsabilités	12
2.9 Contrat de l'API	12
2.10 Description du modèle relationnel	13
2.11 Justification des choix et compromis	13
2.12 Menaces étudiées et protections associées	14
2.13 Déploiement et frontière de confiance	15
2.14 Scénario d'utilisation et séquence métier	16
2.15 Séquence d'enregistrement d'un bagage	16
2.16 Conclusion du chapitre	17

Chapitre 3 : Réalisation, tests et validation	18
3.1 Introduction	18
3.2 Préparation de l'environnement	18
3.3 Fondations : construire et vérifier la chaîne API-base de données	18
3.3.1 Modélisation et premier démarrage	18
3.3.2 Distinguer lecture publique et opérations authentifiées	18
3.3.3 Rendre l'exécution reproductible avec Docker	21
3.4 Interfaces Angular : réaliser un parcours métier complet	22
3.4.1 Préparer les accès public et agents	22
3.4.2 Organiser le travail du superviseur	22
3.4.3 Enregistrer le bagage puis effectuer le tri	23
3.4.4 Vérifier la continuité avec le suivi passager	26
3.4.5 Conserver la trace des corrections exceptionnelles	26
3.4.6 Isoler l'administration et contrôler l'origine de l'interface	26
3.5 Sécurisation réseau : ajouter le VPN aux autorisations applicatives	28
3.5.1 Segmenter les conteneurs et limiter les publications	28
3.5.2 Tester le tunnel avec le même jeton	28
3.6 Kubernetes : déployer les composants et vérifier la persistance	30
3.6.1 Installer le cluster et déclarer les ressources	30
3.6.2 Retrouver les protections réseau et conserver les données	30
3.6.3 Sortir du client Docker : Windows et Ubuntu VMware	33
3.6.4 Préparer une VM dédiée aux validations	34
3.6.5 Vérifier les ports du poste partagé	35
3.7 Recette et premier accès Internet	36
3.7.1 Consulter le suivi par HTTPS et vérifier la restauration	36
3.7.2 Valider WireGuard au travers d'un relais UDP public	36
3.8 Hébergement OVH : rendre la démonstration indépendante du poste	39
3.8.1 Publier le service passager avec un certificat reconnu	39
3.8.2 Contrôler la frontière privée depuis Internet	39
3.8.3 Assurer la maintenance et vérifier la reprise	41
3.9 Bilan de validation et correspondance avec le cahier des charges	41
3.10 Analyse des résultats et portée des tests	43
3.11 Apports du travail et compétences mobilisées	43
3.11.1 Décisions de conception et enseignements	44
3.11.2 Livrables et autonomie de reproduction	44
3.12 Limites et perspectives	44
3.13 Conclusion du chapitre	45
Conclusion générale	46
Références bibliographiques	47

Liste des figures

1	Cas d'utilisation regroupés par responsabilité et périmètre d'accès	8
2	Architecture applicative proposée et réalisée : séparation des interfaces et contrôle des opérations	9
3	Architecture réseau OVH : accès public HTTPS et accès privé par WireGuard	9
4	Organisation Kubernetes : services privés, configuration et persistance sur un seul nœud	10
5	Modèle relationnel simplifié et cardinalités issues des relations de l'application	11
6	Séquence nominale d'enregistrement après authentification et accès VPN	16
7	Routes exposées par Swagger	19
8	Disponibilité de la connexion PostgreSQL	19
9	Refus d'une requête sans JWT	20
10	Suivi public fourni par l'API	20
11	Services de fondation dans Docker Desktop	21
12	Exécution de l'API dans un conteneur	22
13	Suivi et historique depuis Docker	22
14	Accueil public du suivi passager : formulaire à gauche et suite de la page à droite	23
15	Redirection vers la connexion agents	24
16	Tableau de bord du superviseur	24
17	Enregistrement et reçu de suivi	25
18	Transition effectuée par l'agent de tri	25
19	Consultation du statut Trié	26
20	Correction exceptionnelle et historique	27
21	Création d'un compte par l'administrateur	27
22	Contrôle applicatif de l'origine	28
23	Organisation des conteneurs réseau	29
24	Accès privé, coupure et rétablissement du tunnel	29
25	Client extérieur au VPN dans le laboratoire	30
26	Cluster k3s dans k3d	31
27	Ressources observées avec kubectl	31
28	WireGuard et services privés du cluster	32
29	Restrictions et persistance Kubernetes	32
30	Validation avec le client Windows natif	33
31	Validation depuis Ubuntu VMware	33
32	Premier démarrage de la VM Ubuntu	34
33	Contrôles réseau et VPN dans la VM	34
34	HTTPS public depuis la VM dédiée	35
35	Ports LAN fermés et VPN local	36
36	Suivi public par URL Internet temporaire	37
37	Restauration logique vérifiée	37
38	WireGuard Internet avec relais public	38

39	Restrictions et refus d'un pair inconnu	38
40	Application publique hébergée sur OVH	39
41	Suivi du bagage fictif sur OVH	40
42	Scan Internet du VPS sans VPN	40
43	Interface agents atteinte par WireGuard OVH	41

Liste des tableaux

1	Critères d'acceptation et preuves attendues	3
2	Organisation des activités et livrables sur six semaines	5
3	Acteurs et responsabilités dans l'application	6
4	Principaux points d'entrée et protections	12
5	Entités et liens du modèle métier	13
6	Choix techniques, alternatives et compromis	13
7	Lien entre menaces, protections et validations	14
8	Contrôles OVH réussis enregistrés le 2 octobre 2026	42
9	Traçabilité des exigences et de leur validation	42
10	Interprétation des validations et limites de leurs conclusions	43

Introduction générale

La gestion des bagages associe plusieurs opérations successives, de l'enregistrement à la livraison. La qualité du suivi dépend de la cohérence des changements d'état et de la possibilité d'identifier les interventions des agents. Une modification non autorisée ou une rupture de traçabilité limite la fiabilité des informations communiquées au passager.

Dans ce contexte, le projet étudie la conception d'un système qui associe gestion du parcours et sécurisation de l'infrastructure. La problématique est de fournir un suivi public utile tout en réservant les fonctions internes aux utilisateurs habilités et à un réseau autorisé. La réalisation prend la forme d'un démonstrateur : aucune intégration à un système aéroportuaire réel n'est revendiquée.

L'objectif est de permettre au passager de consulter un suivi limité à l'aide d'un code aléatoire, tout en réservant les opérations internes aux agents authentifiés et connectés au VPN. La protection réseau complète les autorisations applicatives : disposer d'un jeton valide ne suffit pas à atteindre le service privé hors du réseau autorisé.

Le rapport est organisé en trois chapitres. Le premier présente le contexte, la mission, les objectifs, les besoins et l'organisation du travail. Le deuxième traduit ces besoins en une architecture applicative et réseau. Le troisième décrit la réalisation progressive et les validations, avec les captures associées aux étapes correspondantes.

Chapitre 1 : Étude préalable et analyse des besoins

1.1 Introduction

L'étude préalable situe la mission et précise le problème auquel la réalisation doit répondre. Elle conduit à définir les objectifs, à examiner les limites d'une gestion sans séparation des accès, puis à proposer une solution cohérente. L'identification des acteurs et des besoins complète cette démarche avant la conception technique.

1.2 Contexte et définition de la mission

Le projet s'inscrit dans une démarche de stage orientée vers l'administration des réseaux et la sécurité des infrastructures. La mission consiste à concevoir un démonstrateur de gestion des bagages et à sécuriser son déploiement. Elle associe le développement d'une application fonctionnelle à des vérifications de contrôle d'accès, de segmentation réseau et de persistance.

Le parcours du bagage implique plusieurs responsabilités : enregistrement, manutention, supervision et information du passager. Chaque acteur doit disposer des fonctions nécessaires à son travail sans accéder aux opérations réservées aux autres. L'administrateur gère les comptes au moyen d'une identité dédiée. Aucun organisme d'accueil ni système aéroportuaire réel n'est attribué à cette réalisation ; le périmètre décrit est celui du démonstrateur.

1.3 Problématique

La question centrale est la suivante : comment assurer le suivi d'un bagage tout en empêchant les utilisateurs publics d'atteindre les fonctions internes, même s'ils disposent d'un jeton applicatif valide ? Une seconde difficulté concerne la traçabilité : les changements d'état doivent rester compréhensibles, y compris lorsqu'une correction exceptionnelle intervient.

Ces deux dimensions imposent de traiter ensemble les règles métier et la sécurité de l'infrastructure. Un formulaire de connexion ne suffit pas à créer une frontière réseau, tandis qu'un VPN ne décide pas du rôle autorisé à modifier un bagage.

1.4 Objectifs à atteindre

La réalisation vise à fournir un suivi public limité au statut et aux horodatages, à enregistrer les bagages associés aux vols et à contrôler leurs transitions. Elle doit conserver les interventions des

agents et les motifs des corrections. Sur le plan technique, elle doit limiter les opérations internes au VPN, protéger les échanges par HTTPS et maintenir PostgreSQL dans le réseau privé.

Les livrables attendus comprennent les sources, les configurations Docker et Kubernetes, les scripts de validation, les guides de reproduction et le rapport. Les résultats doivent être observables au moyen de scénarios métier, de contrôles réseau et de captures de l'application réellement exécutée.

1.5 Critères d'acceptation du démonstrateur

Une exigence est déclarée validée lorsqu'un scénario identifié produit le résultat attendu et qu'une preuve consigne le contexte d'exécution. Une page affichée ou un conteneur démarré ne suffit donc pas à valider la sécurité. Les critères du tableau 1 servent de référence à la recette du troisième chapitre.

TABLE 1 – Critères d'acceptation et preuves attendues

Exigence	Condition de réussite	Preuve attendue
Suivi public	Un code fictif connu retourne le statut et les horodatages sans identité du passager; les routes privées ne sont pas relayées.	Réponses HTTP et consultation publique.
Autorisations	L'action permise aboutit; une requête sans JWT est refusée et un rôle insuffisant ne modifie pas les données.	Recette métier et contrôles 401/403.
Frontière VPN	Avec le même JWT encore valide, l'accès privé fonctionne avec le tunnel, devient inaccessible à sa coupure, puis revient après rétablissement.	Série VPN actif, coupé, rétabli depuis un client identifié.
Cycle et historique	Une transition autorisée change le statut et ajoute un événement daté lié à l'agent; une correction nécessite le rôle et le motif prévus.	Réponses métier et historique consulté.
Base privée	Les ports sensibles documentés ne sont pas accessibles depuis les clients extérieurs testés, en IPv4 et IPv6 sur OVH.	Résultats des sondes et configuration des services.
Persistance et reprise	Les données contrôlées restent présentes après remplacement du pod et redémarrage; un dump peut être restauré dans une base distincte.	Comparaison avant/après et compte rendu de restauration.

Les critères définissent un périmètre de démonstration. La disponibilité sous forte charge, la perte complète d'un disque et une résistance à toutes les attaques ne sont pas déduites de ces vérifications.

1.6 Analyse et critique de l'existant

L'analyse porte ici sur un scénario de référence pédagogique : une application où les opérations internes et le suivi passager seraient accessibles par le même point d'entrée, avec une protection reposant principalement sur la connexion utilisateur. Il ne s'agit pas d'un audit d'un système d'aéroport existant.

Dans ce scénario, le masquage des actions dans l'interface ne garantit pas leur protection face à des appels directs à l'API. Une exposition de la base de données élargit aussi la surface accessible. Enfin, une mise à jour du statut sans conservation de l'agent, de l'heure et du motif rend les anomalies difficiles à expliquer. Ces limites motivent la séparation des services, l'autorisation côté API et l'historique des transitions.

1.7 Solution proposée

La solution distingue une application publique de suivi et une application privée destinée aux agents. Le proxy public relaie exclusivement le suivi en lecture. Les fonctions internes nécessitent un chemin réseau WireGuard, une authentification JWT valide et un rôle approprié. PostgreSQL conserve les données métier et l'historique dans un segment privé.

La mise en œuvre commence dans un environnement local reproductible, puis évolue vers Kubernetes et un VPS OVH. Les vérifications accompagnent cette progression : chaque frontière ajoutée doit être testée, et chaque changement de déploiement doit préserver les fonctions déjà réalisées.

1.8 Méthodologie et organisation du travail

Le travail suit une démarche incrémentale. Chaque étape possède un objectif, des livrables et une condition de validation. La base et l'API précèdent les interfaces ; les parcours métier précèdent les tests réseau ; le déploiement local précède l'hébergement distant. Les guides consignent les commandes et les limites, tandis que les captures servent de preuves visuelles.

Cette organisation reprend la progression étude, analyse, conception, réalisation et validation du rapport de licence fourni comme modèle. Elle ne suppose pas l'existence d'une équipe Scrum ou de cérémonies qui n'ont pas été documentées pour ce projet.

1.9 Planning de référence sur six semaines

Le tableau suivant répartit les activités sur six semaines de stage, sans dates calendaires. Il présente une organisation de référence des tâches ; les preuves techniques conservent leurs dates d'exécution propres et ne sont pas réattribuées artificiellement à ces semaines.

TABLE 2 – Organisation des activités et livrables sur six semaines

Période	Activités	Livrables et jalon
Semaine 1	Cadrage, analyse des besoins, acteurs, règles métier et conception du modèle de données.	Cahier des charges, modèle relationnel et architecture cible.
Semaine 2	API .NET, migrations PostgreSQL, JWT, rôles, historique et premiers conteneurs.	API reliée à la base; suivi public et contrôles d'autorisation vérifiés.
Semaine 3	Applications Angular, parcours d'enregistrement et de tri, suivi passager et administration.	Interfaces intégrées; parcours métier et corrections motivées vérifiés.
Semaine 4	Segmentation Docker, proxys public/privé, TLS, WireGuard et durcissement.	Accès privé avec VPN; coupure et refus hors VPN contrôlés.
Semaine 5	Kubernetes, stockage persistant, Network-Policy et essais depuis Windows et VM-ware.	Déploiement reproductible; persistance et protections réseau vérifiées.
Semaine 6	Préparation de l'hébergement, recette finale, sauvegarde/restauration et rédaction.	Dossier de validation, captures légendées et rapport; hébergement OVH documenté comme prolongement de démonstration.

Le planning rend visible la dépendance entre les activités plutôt qu'un découpage indépendant de tâches. La documentation et les vérifications accompagnent toutes les semaines. La préparation de l'hébergement est distinguée de sa validation distante effective, détaillée dans le chapitre de réalisation.

1.10 Présentation du projet

Le système couvre l'enregistrement, le tri, le chargement, le transport et la livraison des bagages. Les données de démonstration sont fictives. Le projet ne revendique aucune intégration à un système réel d'aéroport ou à un dispositif matériel de lecture.

1.11 Acteurs et responsabilités

1.12 Besoins fonctionnels

Le système doit enregistrer les bagages associés aux vols, contrôler les transitions de statut, conserver chaque événement et fournir un suivi public qui ne révèle pas l'identité du passager. Les corrections

TABLE 3 – Acteurs et responsabilités dans l'application

Acteur	Responsabilité
Passager	Consultation du statut et des horodatages avec un code de suivi.
Agent d'enregistrement	Enregistrement et consultation des bagages selon les permissions.
Agent de tri	Transitions de manutention autorisées.
Superviseur	Gestion des vols et corrections exceptionnelles motivées.
Administrateur	Gestion des comptes, sans héritage automatique des droits métier.

exceptionnelles nécessitent une autorisation spécifique et un motif.

1.13 Besoins de sécurité

Les opérations privées exigent simultanément un accès réseau autorisé, une authentification valide et un rôle compatible avec l'action. La base de données ne doit pas être exposée publiquement. Les mots de passe sont hachés ; les jetons sont limités dans le temps. La sauvegarde et la restauration doivent être vérifiées.

1.14 Traçabilité et sécurité des données

La traçabilité repose sur la conservation des changements d'état d'un bagage. Chaque événement permet de comprendre le parcours enregistré et les interventions des agents. Le suivi public est volontairement limité : il expose le statut et les horodatages, sans afficher l'identité du passager. Le code de suivi doit lui-même être conservé de manière confidentielle par son détenteur.

1.15 Authentification et contrôle d'accès

L'authentification identifie le compte qui effectue une requête. L'autorisation détermine ensuite si son rôle permet l'opération. Un jeton JWT signé transmet des informations d'identité vérifiées par l'API. Le format JWT [1] est défini par la RFC 7519. Dans ce projet, sa durée de validité est de quinze minutes ; la signature, l'émetteur, le destinataire, l'expiration et l'état du compte sont contrôlés.

Le contrôle par rôle distingue l'enregistrement, le tri, la supervision et l'administration. L'administrateur ne possède pas automatiquement les permissions métier. Le masquage des boutons dans l'interface facilite l'usage, mais l'API reste responsable de la décision d'autorisation.

1.16 TLS et réseau privé virtuel

TLS protège les échanges HTTPS entre le client et le service. Le certificat public du VPS est reconnu par les clients testés. L'interface agents utilise une autorité de certification privée installée sur le poste de démonstration.

WireGuard [2] encapsule les paquets IP dans un tunnel chiffré utilisant UDP. Dans le projet, les pairs configurés accèdent au réseau privé par ce tunnel. Le tunnel ne remplace pas l'authentification applicative : un client connecté doit encore présenter un jeton valide et disposer du rôle approprié. Inversement, un jeton valide n'ouvre pas un chemin réseau vers les fonctions internes en l'absence du tunnel.

1.17 Conteneurisation et orchestration

Docker [3] exécute les composants dans des conteneurs, c'est-à-dire des processus isolés accompagnés de leurs fichiers nécessaires. Les NetworkPolicies de Kubernetes [4] permettent de définir les communications autorisées entre les pods. Kubernetes organise aussi les déploiements, les services et la persistance. La réalisation utilise k3s dans k3d, avec un seul nœud. Les politiques réseau autorisent seulement les flux nécessaires entre les proxys, l'API, la base et le DNS. Cette configuration améliore la séparation des composants sans apporter de haute disponibilité.

1.18 Conclusion du chapitre

L'analyse des besoins conduit à distinguer deux axes complémentaires : la gestion fiable du parcours métier et la limitation des accès aux services internes. Le chapitre suivant présente leur traduction dans l'architecture et le modèle de données.

Chapitre 2 : Conception de l'architecture proposée

2.1 Cas d'utilisation et responsabilités

Le diagramme de la figure 1 associe chaque acteur à son groupe de fonctions. Les opérations internes exigent le VPN et une session autorisée. Le superviseur peut enregistrer des bagages et effectuer des transitions; sa responsabilité particulière porte sur les vols et les corrections motivées. L'administrateur reste séparé des opérations métier.

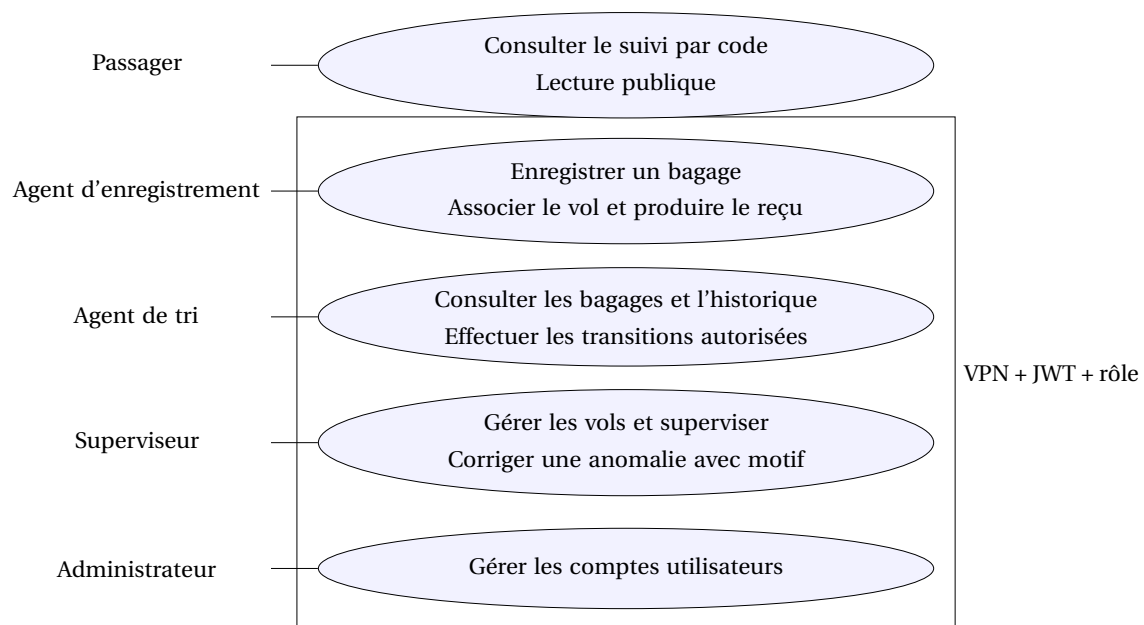


FIGURE 1 – Cas d'utilisation regroupés par responsabilité et périmètre d'accès

2.2 Architecture générale

Angular [5] fournit le cadre de développement des interfaces web. Dans notre architecture, deux applications Angular sont servies par des proxys Nginx distincts. Le proxy public autorise uniquement le suivi en lecture. Le proxy agents est accessible par WireGuard et transmet les opérations privées à l'API. PostgreSQL reste dans le réseau interne.

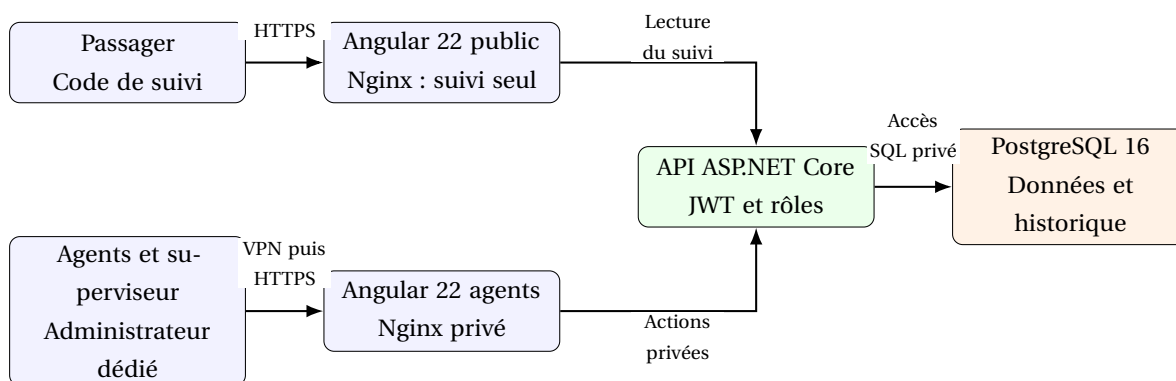


FIGURE 2 – Architecture applicative proposée et réalisée : séparation des interfaces et contrôle des opérations

La figure 2 présente les responsabilités des composants. Le site passager utilise uniquement le suivi public, tandis que les agents accèdent aux opérations internes. Les deux chemins aboutissent à l'API, qui applique les règles métier avant toute interaction avec PostgreSQL. Angular fournit les interfaces utilisateur et PostgreSQL assure la persistance relationnelle.

2.3 Architecture réseau de l'hébergement OVH

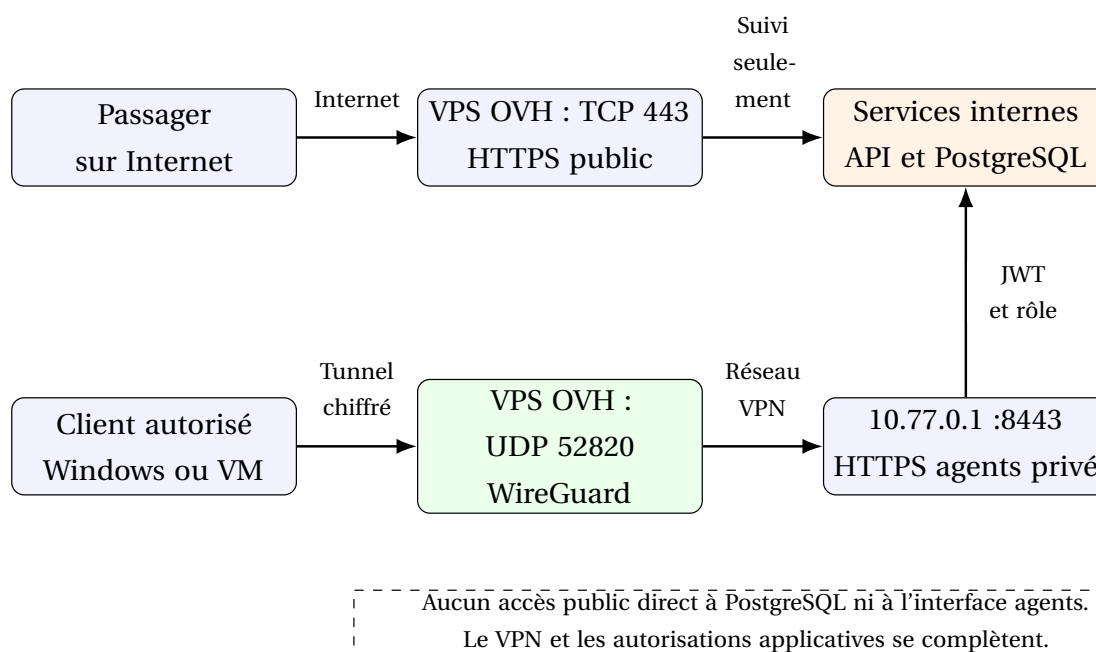


FIGURE 3 – Architecture réseau OVH : accès public HTTPS et accès privé par WireGuard

La figure 3 distingue les connexions publiques et le tunnel privé. WireGuard transporte les paquets réseau chiffrés au-dessus d'UDP. Le port 8443 appartient à l'adresse du VPN : il n'est pas publié comme service privé directement accessible depuis Internet. L'administration SSH par clé constitue un accès

de maintenance distinct.

2.4 Architecture Kubernetes et stockage

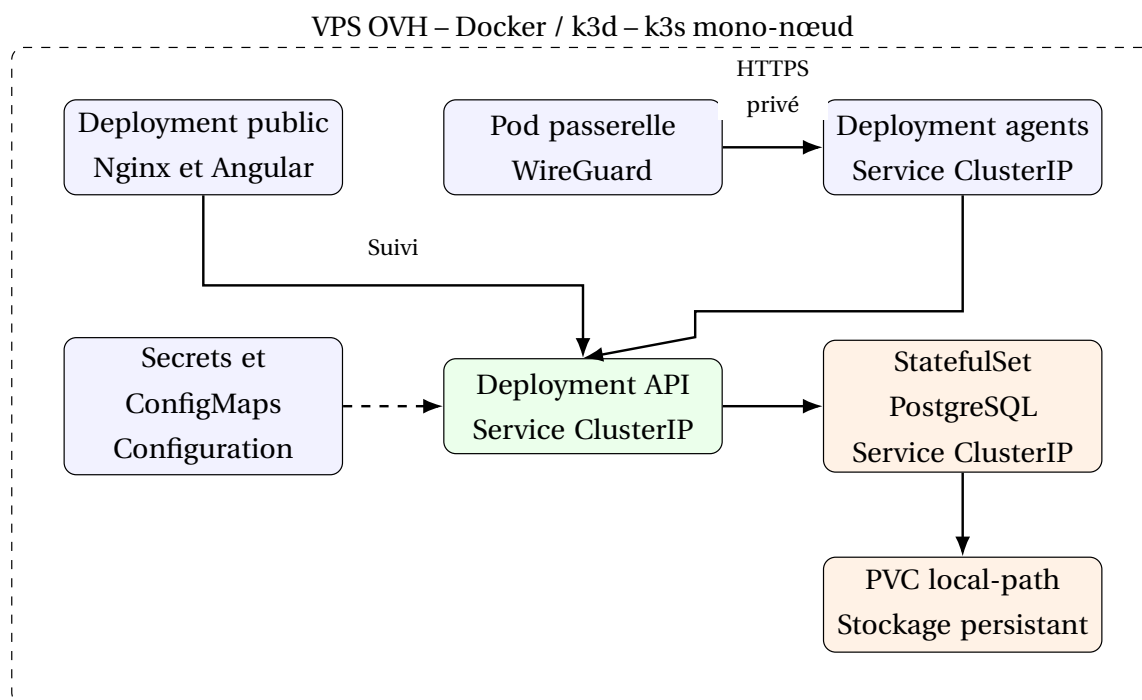


FIGURE 4 – Organisation Kubernetes : services privés, configuration et persistance sur un seul nœud

La figure 4 montre le déploiement de démonstration. Les NetworkPolicies restreignent les flux aux communications explicitement autorisées. Les migrations et l'initialisation sont réalisées par des Jobs. Le volume conserve les données après remplacement du pod PostgreSQL, mais son stockage local ne garantit pas la reprise après perte du nœud.

2.5 Modèle de données

PostgreSQL [6] est le système de gestion de base de données relationnelle utilisé pour cette réalisation. Les tables principales sont `roles`, `users`, `vols`, `bagages` et `historique_statuts`. Un bagage appartient à un vol. Les événements d'historique conservent l'état, la date et l'agent responsable. Le statut et son événement sont enregistrés atomiquement; le contrôle de concurrence évite les écrasements silencieux.

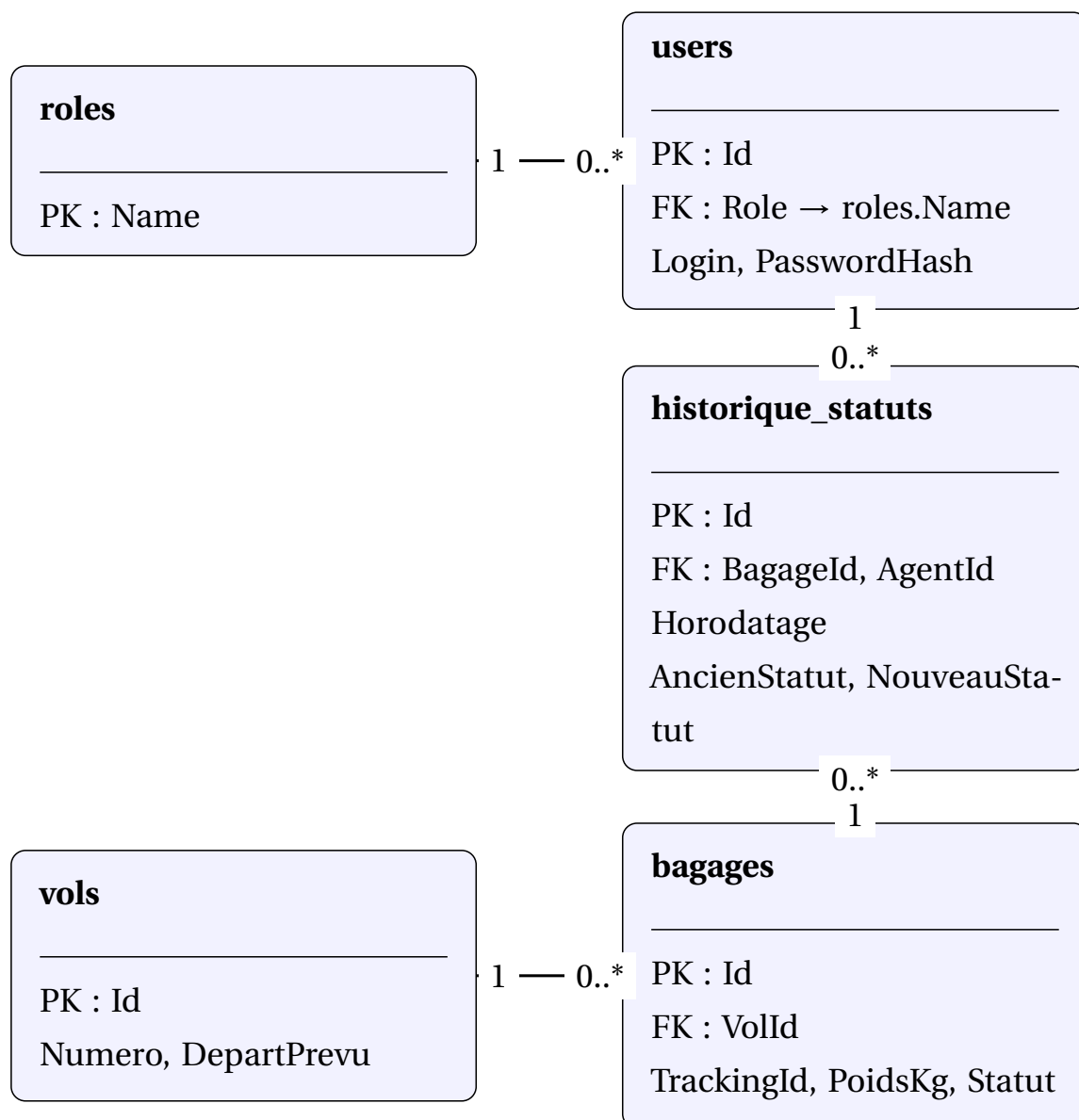


FIGURE 5 – Modèle relationnel simplifié et cardinalités issues des relations de l'application

Le schéma présente les clés et les liens nécessaires à la compréhension du parcours, sans détailler toutes les colonnes. Un compte possède un rôle; chaque bagage appartient à un vol et chaque événement référence un bagage et un agent. Les relations restrictives évitent de supprimer une entité encore référencée. L'unicité du code de suivi distingue chaque bagage.

2.6 Cycle de vie d'un bagage

Le parcours normal est : enregistré, trié, chargé, en vol, livré. Une déclaration de perte interrompt ce parcours. La récupération d'un bagage perdu relève d'une correction motivée du superviseur.

2.7 Séparation des protections

Le VPN contrôle l'accès réseau; JWT identifie le compte; les rôles autorisent les opérations. Les NetworkPolicies appliquent un refus par défaut puis autorisent les flux nécessaires. Le compte PostgreSQL applicatif peut lire et ajouter l'historique, sans le modifier ni le supprimer. Le propriétaire de la base reste un acteur de confiance.

2.8 Organisation du code et des responsabilités

Le backend regroupe les contrats HTTP, les règles métier, l'accès aux données et les contrôles de sécurité. Entity Framework Core décrit les relations et exécute les migrations PostgreSQL. Les applications Angular publique et agents sont compilées séparément afin que le service passager n'embarque pas les écrans internes. Les scripts de démarrage et de validation organisent les expériences; les manifests déclarent les ressources Kubernetes.

2.9 Contrat de l'API

TABLE 4 – Principaux points d'entrée et protections

Route	Usage et contrôle
GET /api/track/{code}	Lecture publique du statut et des événements, sans identité du passager.
POST /api/auth/login	Connexion privée, limitation du débit et verrouillage après échecs.
GET /api/vols	Consultation autorisée dans le réseau privé.
POST /api/vols	Création réservée au superviseur.
POST /api/bagages	Enregistrement par un rôle habilité.
GET /api/bagages	Liste et filtres selon les permissions.
PATCH /api/bagages/{id}/statut	Transition normale ou correction motivée, contrôlée par l'API.
GET /api/bagages/{id}/historique	Consultation privée de la traçabilité.

Les réponses distinguent notamment l'absence d'authentification (401), un rôle insuffisant (403), une ressource absente (404), un conflit métier ou concurrent (409) et un débit dépassé (429). Le suivi public ne relaie pas les fonctions de connexion, comptes, vols ou modification des bagages.

2.10 Description du modèle relationnel

TABLE 5 – Entités et liens du modèle métier

Entité	Rôle dans le modèle
roles	Définit les permissions attribuées aux comptes.
users	Compte, mot de passe haché et rôle associé.
vols	Référence du vol auquel les bagages sont rattachés.
bagages	Données d'enregistrement, vol, statut courant et code de suivi aléatoire.
historique_statuts	Événements datés liés au bagage et à l'agent responsable.

La séparation du statut courant et des événements permet une consultation rapide tout en conservant le parcours. Le contrôle de concurrence utilise `xmin` PostgreSQL. Une modification concurrente ne doit pas effacer silencieusement une autre transition. Les droits SQL de l'historique limitent le compte applicatif à la lecture et à l'ajout.

2.11 Justification des choix et compromis

Les choix sont appréciés au regard de trois contraintes : réaliser un parcours métier complet, rendre les protections vérifiables et conserver un environnement reproductible sur le poste de travail puis sur un VPS. Aucune comparaison de performances entre produits n'a été menée; le tableau 6 présente les raisons de conception, plutôt qu'un classement des technologies.

TABLE 6 – Choix techniques, alternatives et compromis

Choix	Alternative possible	Raison dans ce projet	Compromis
WireGuard	API privée publiée avec authentification seule; autre VPN.	Ajouter une frontière réseau indépendante du JWT et vérifier sa coupure depuis Windows et Ubuntu.	Configuration et distribution des clés des pairs.
PostgreSQL	Autre base relationnelle; base documentaire.	Les liens bagage-vol-agent et l'historique justifient un modèle relationnel; les contraintes et transactions soutiennent la cohérence métier.	Migrations et administration de la base nécessaires.

Choix	Alternative possible	Raison dans ce projet	Compromis
Docker puis Kubernetes	Exécution native; maintien sous Compose.	Compose stabilise d'abord l'exécution; Kubernetes permet ensuite d'étudier services privés, politiques réseau et stockage persistant.	Complexité supérieure à Compose pour un petit démonstrateur.
Deux interfaces Angular	Une interface unique avec menus conditionnels.	Distinguer les usages public et agents ainsi que leurs points d'entrée; garder les décisions d'autorisation dans l'API.	Deux compilations et deux configurations à maintenir.
VPS OVH	Démonstration locale ou dans une VM.	Disposer d'un point d'entrée public pour l'hébergement HTTPS et les validations du VPN depuis Internet.	Abonnement actif et administration de l'hôte.

Kubernetes n'est pas indispensable au seul suivi d'un bagage. Son intérêt ici est pédagogique et architectural : appliquer la segmentation et vérifier la persistance dans un environnement orchestré. Le choix d'un seul nœud reste proportionné à la démonstration, mais ne fournit pas de haute disponibilité. De même, séparer deux interfaces facilite le déploiement; la sécurité dépend toujours des proxys, du VPN et des autorisations côté serveur.

2.12 Menaces étudiées et protections associées

Les biens à protéger sont les données des bagages, l'historique, les comptes et les secrets d'accès. Les points d'entrée considérés sont le site public, le service VPN et les routes privées de l'API. L'analyse porte sur un utilisateur extérieur, un utilisateur possédant un JWT et un agent dont le rôle ne permet pas l'action demandée. L'administrateur de l'hôte demeure dans le périmètre de confiance.

TABLE 7 – Lien entre menaces, protections et validations

Menace	Protection retenue	Vérification	Risque résiduel
Accès privé depuis Internet	Proxy public limité au suivi; services privés; VPN obligatoire.	Routes privées re-fusées et coupure du tunnel avec le même JWT.	Une mauvaise configuration future peut élargir l'exposition.
Réutilisation d'un JWT obtenu par un tiers	Signature, durée limitée, contrôles de compte et rôle; accès réseau privé distinct.	JWT seul insuffisant hors VPN; contrôles d'authentification.	JWT et accès VPN compromis ensemble; poste client compromis.

Menace	Protection retenue	Vérification	Risque résiduel
Modification par un rôle non habilité	Autorisation dans l'API et compte administrateur distinct des rôles métier.	Actions permises et interdites dans la recette.	Compromission d'un compte effectivement habilité.
Fuite de données par le suivi	Réponse publique limitée et code aléatoire; absence d'identité du passager.	Lecture des réponses publiques et consultation de l'interface.	Divulgaration du code donnant accès au suivi limité.
Altération du parcours	Transitions contrôlées, motif pour correction, événement associé et contrôle de concurrence.	Recette du cycle et consultation de l'historique.	Contrôle complet de la base par son propriétaire.
Accès direct à PostgreSQL	Réseau privé, service non publié, politiques réseau et droits SQL limités.	Sondes extérieures sur les ports documentés.	Compromission de l'hôte; scan non exhaustif.
Perte des données	Volume persistant et sauvegarde logique avec restauration vérifiée.	Remplacement du pod, redémarrage et restauration.	Perte du disque; copie hors VPS encore manuelle.

Ces protections se complètent. Le VPN contrôle le chemin d'accès; le JWT identifie une session; le rôle décide de l'opération autorisée. L'historique permet ensuite d'expliquer la modification. Aucun de ces mécanismes, pris isolément, ne démontre la sécurité de l'ensemble. Le tableau conserve donc les risques résiduels au lieu d'assimiler une validation ponctuelle à l'absence de risque.

2.13 Déploiement et frontière de confiance

Dans Docker Compose, les réseaux edge, public_api, private_api et database limitent les communications. Dans Kubernetes, les services API, agents et base sont de type ClusterIP. Des Deployments exécutent l'API et les interfaces; un StatefulSet et un volume persistant portent PostgreSQL; des Jobs réalisent les migrations et l'initialisation.

Les secrets de base de données, de signature JWT et de VPN sont générés hors des images puis fournis par des Secrets. Les conteneurs applicatifs utilisent des comptes non privilégiés, des fichiers en lecture seule et aucun jeton d'administration Kubernetes. L'administrateur de l'hôte et le propriétaire de la base demeurent des acteurs de confiance : ces mécanismes ne protègent pas contre leur contrôle complet du système.

2.14 Scénario d'utilisation et séquence métier

Le superviseur crée un vol. L'agent d'enregistrement sélectionne ce vol et saisit un bagage fictif; l'API crée le bagage et son premier événement, puis fournit un code de suivi. L'agent de tri consulte la liste, choisit le bagage et demande une transition autorisée. L'API contrôle le rôle, le statut précédent et la concurrence avant d'enregistrer l'événement. Le passager utilise enfin son code sur le site public pour lire le statut et les horodatages.

2.15 Séquence d'enregistrement d'un bagage

La figure 6 détaille le scénario après connexion et établissement du VPN. Les contrôles sont effectués avant l'écriture : la présence d'un formulaire ne suffit pas à autoriser l'enregistrement. Le bagage et son premier événement sont enregistrés dans une même opération de persistance.

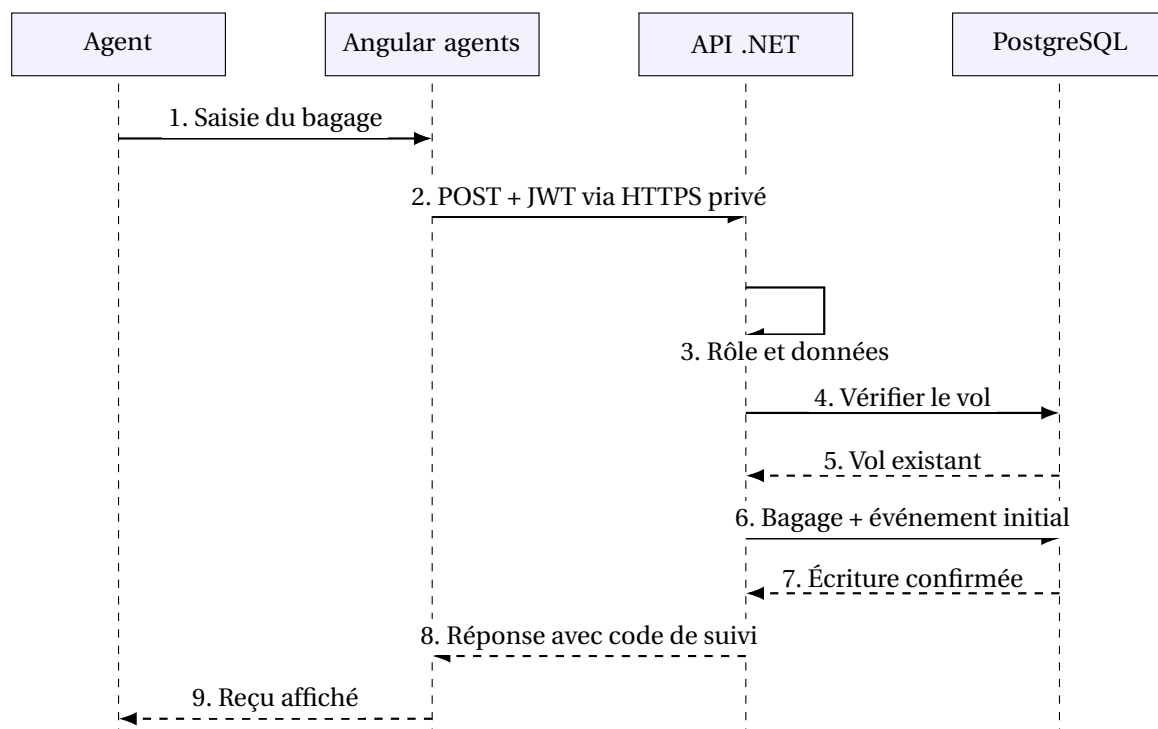


FIGURE 6 – Séquence nominale d'enregistrement après authentification et accès VPN

Un rôle insuffisant, un vol absent ou des données invalides interrompent ce scénario avant la création. Le passager consulte ensuite le code sur l'application publique; il ne participe pas à l'enregistrement privé. Cette distinction relie la conception aux captures du reçu, du tri et du suivi présentées dans la réalisation.

2.16 Conclusion du chapitre

La conception sépare les responsabilités des utilisateurs, les chemins réseau et les privilèges des composants. La réalisation suivante vérifie ces choix par des scénarios métier et par des tentatives d'accès depuis différents périmètres.

Chapitre 3 : Réalisation, tests et validation

3.1 Introduction

La réalisation a été conduite progressivement : construire d'abord une API reliée à PostgreSQL, développer les parcours Angular, protéger les services privés, puis déployer la solution sur Kubernetes et sur un VPS. À chaque étape, les observations ont orienté le travail suivant. Les captures sont insérées au moment où elles éclairent une décision ou un résultat ; elles ne constituent pas une suite indépendante de la démarche.

3.2 Préparation de l'environnement

Le poste Windows accueille les sources, Docker Desktop et les outils de validation. La pile retenue associe .NET 8, Entity Framework Core, PostgreSQL 16 et Angular 22. La virtualisation VMware permet ensuite de vérifier la solution depuis Ubuntu. Les comptes, vols et passagers utilisés sont fictifs. Les résultats présentés sont ceux enregistrés les 1er et 2 octobre 2026.

3.3 Fondations : construire et vérifier la chaîne API–base de données

3.3.1 Modélisation et premier démarrage

La première étape consiste à créer les tables des comptes, rôles, vols, bagages et événements. Les migrations rendent cette structure reproductible. Une fois la connexion PostgreSQL configurée, l'API est démarrée en environnement local ; Swagger donne une vue de son contrat et permet de contrôler les réponses du serveur.

La liste des routes permet de repérer les fonctions disponibles, tandis que la réponse 200 de la sonde de disponibilité confirme la connexion à PostgreSQL. Ces deux observations valident le point de départ technique ; elles ne démontrent pas encore la sécurité des opérations.

3.3.2 Distinguer lecture publique et opérations authentifiées

Le suivi passager doit fonctionner sans compte, mais la consultation des vols relève des opérations internes. Cette différence est vérifiée directement dans l'API : un appel aux vols sans jeton est refusé, puis le code fictif de suivi permet de lire un parcours déjà enregistré.

La figure 9 vérifie le refus sans JWT, tandis que la figure 10 vérifie le maintien de la lecture publique. Le refus 401 est le résultat attendu de l'authentification applicative. La réponse de suivi présente le statut et les événements sans révéler l'identité du passager. La séparation fonctionnelle est donc établie avant l'ajout du VPN.

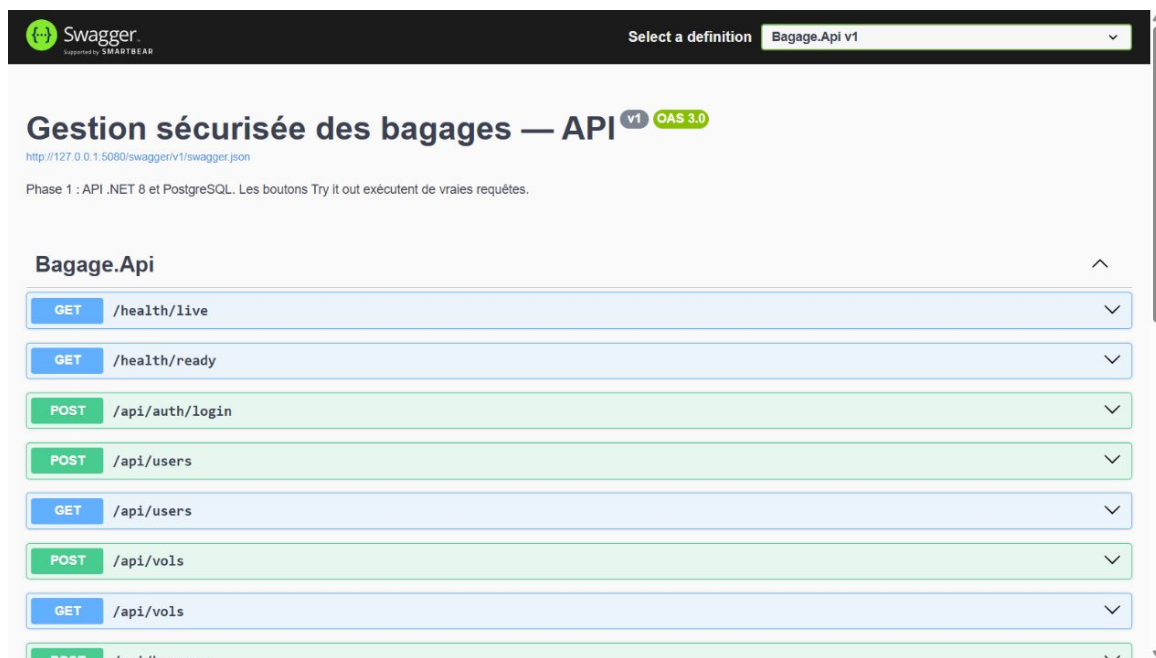


FIGURE 7 – Routes exposées par Swagger

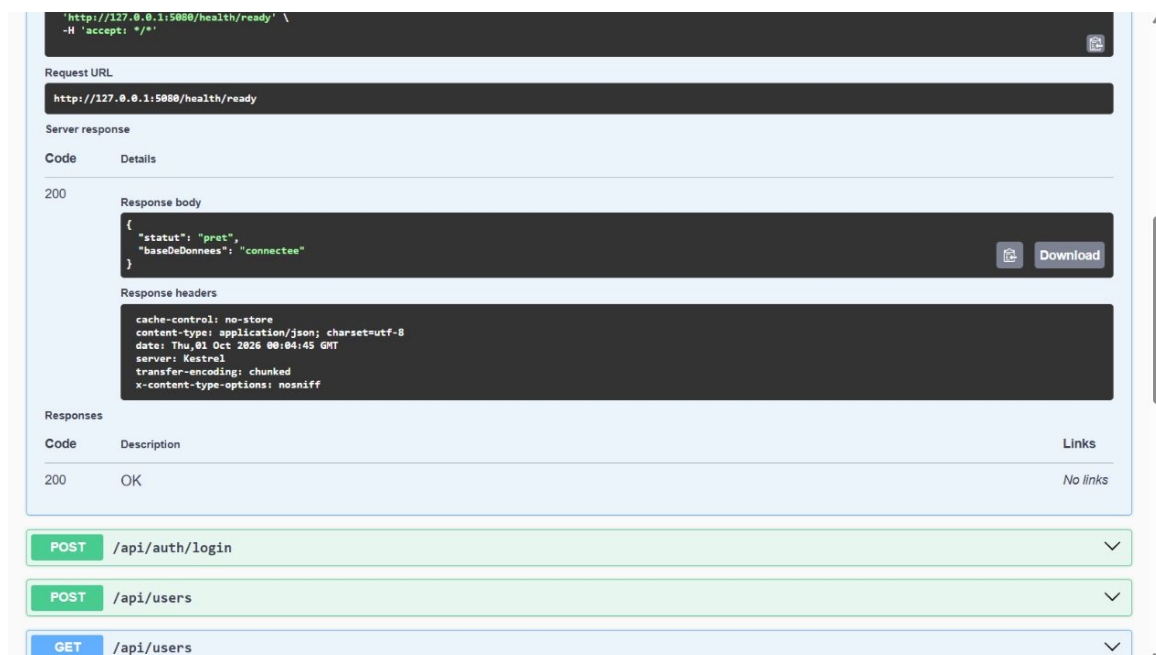


FIGURE 8 – Disponibilité de la connexion PostgreSQL

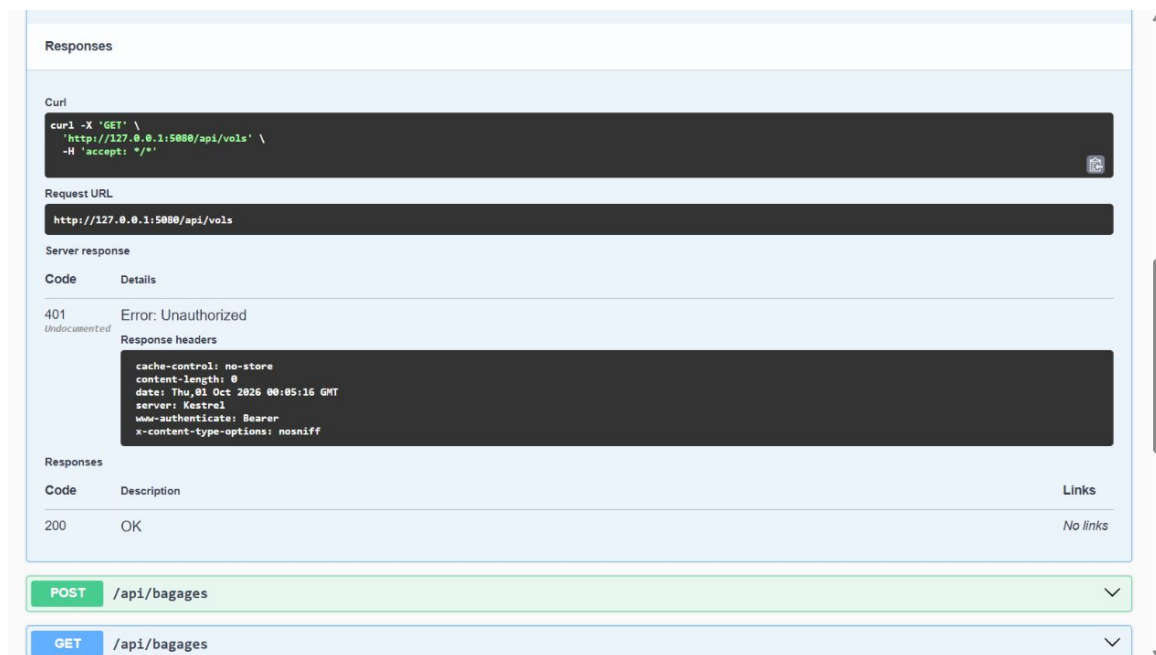


FIGURE 9 – Refus d'une requête sans JWT

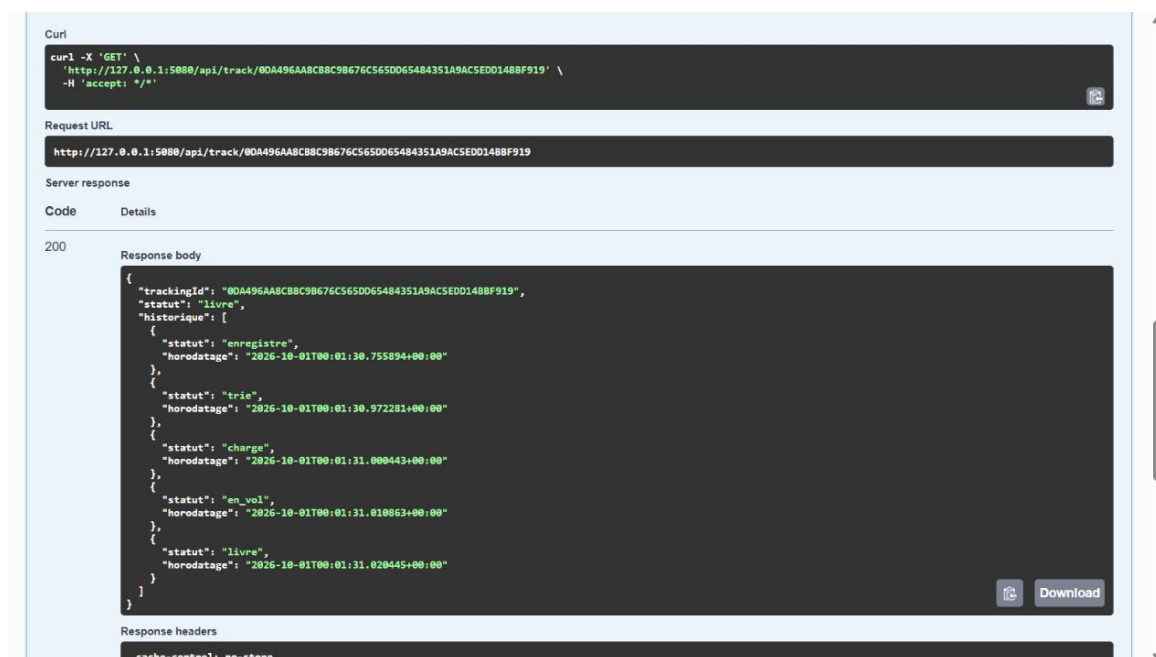


FIGURE 10 – Suivi public fourni par l'API

3.3.3 Rendre l'exécution reproductible avec Docker

L'API et PostgreSQL sont ensuite exécutés dans des conteneurs pour stabiliser leur environnement. Docker Desktop permet d'observer les deux services. La sonde de vie et le suivi sont rejoués sur l'API conteneurisée afin de vérifier que le changement d'environnement conserve le fonctionnement métier.

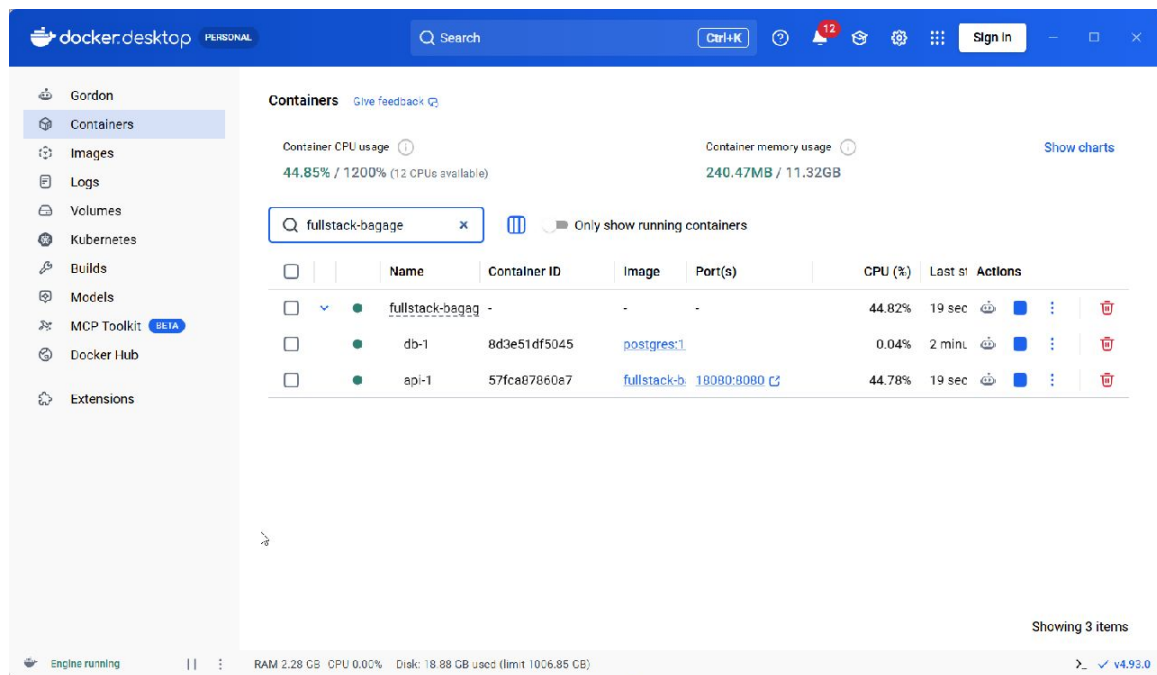


FIGURE 11 – Services de fondation dans Docker Desktop

Les réponses restent cohérentes avec l'exécution native. Le port local 18080 appartient à cette étape de développement; les phases suivantes retireront la publication directe de l'API. La chaîne de données étant opérationnelle, le travail peut se poursuivre sur les interfaces utilisateur.

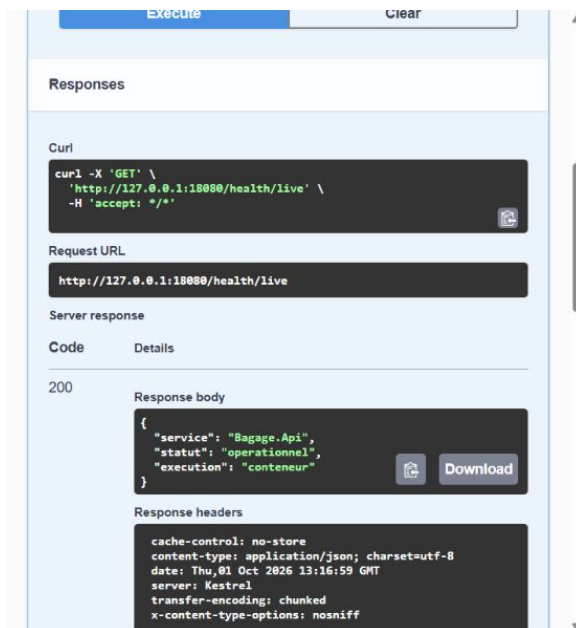


FIGURE 12 – Exécution de l'API dans un conteneur

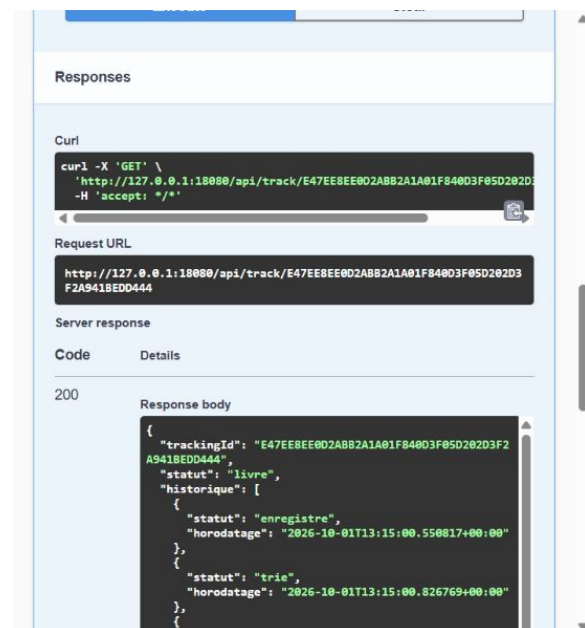


FIGURE 13 – Suivi et historique depuis Docker

3.4 Interfaces Angular : réaliser un parcours métier complet

3.4.1 Préparer les accès public et agents

Deux applications Angular sont compilées séparément. L'accueil public propose uniquement la recherche par code. L'espace agents possède une navigation privée ; lorsque la page des bagages est demandée sans session, la garde conduit vers la connexion. Cette séparation rend visibles les deux usages dès l'entrée dans l'application.

La figure 14 répond au besoin de consultation sans compte ; la figure 15 matérialise le point d'entrée réservé aux agents. Le passager n'a pas à s'authentifier pour utiliser son code. L'agent doit ouvrir une session avant les fonctions internes. Cette protection de navigation accompagne les contrôles de l'API ; elle ne les remplace pas.

3.4.2 Organiser le travail du superviseur

Le superviseur dispose d'une vue métier des bagages et des vols. Les indicateurs du tableau de bord sont calculés à partir des listes chargées et sont présentés avec leurs limites. Ce rôle prépare le contexte de l'enregistrement et ne donne pas automatiquement accès à la gestion des comptes.

La capture montre les menus métier accessibles au superviseur et l'absence du menu comptes. La distinction des responsabilités est ainsi visible dans l'interface avant d'être contrôlée lors des appels API.

bagage. Espace passagers

DU DÉPART À L'ARRIVÉE

Votre voyage. Votre bagage. L'esprit tranquille.

Retrouvez les étapes de votre bagage,
de son enregistrement à sa livraison.

◆ Sans compte. Sans donnée personnelle affichée.

SUIVI DES BAGAGES

Où en est mon bagage ?

Saisissez le code qui vous a été remis lors de l'enregistrement.

Code de suivi

Votre code à 48 caractères

Le code figure sur votre reçu d'enregistrement.

Suivre mon bagage →

© Conservez votre code de suivi à l'abri des regards.

CHAQUE ÉTAPE COMPTE
Un parcours, cinq étapes.

01 Enregistré 02 Trié 03 Chargé 04 En vol 05 Livré

Besoin d'aide ?
Pour un code introuvable ou une question, contactez le comptoir bagages de votre compagnie.

bagage - Suivi aéroportuaire Votre code, votre suivi.

FIGURE 14 – Accueil public du suivi passager : formulaire à gauche et suite de la page à droite

3.4.3 Enregistrer le bagage puis effectuer le tri

Après création du vol, l'agent d'enregistrement sélectionne ce vol et saisit les données fictives du bagage. La création produit un reçu et un code de suivi. L'agent de tri retrouve ensuite le bagage et demande le passage à l'étape Trié; l'API contrôle la transition et ajoute l'événement associé.

Le reçu relie l'opération interne au suivi passager. L'historique de tri montre que le statut n'est pas une simple modification d'affichage : il correspond à une transition conservée en base. Le passager peut maintenant consulter ce parcours.

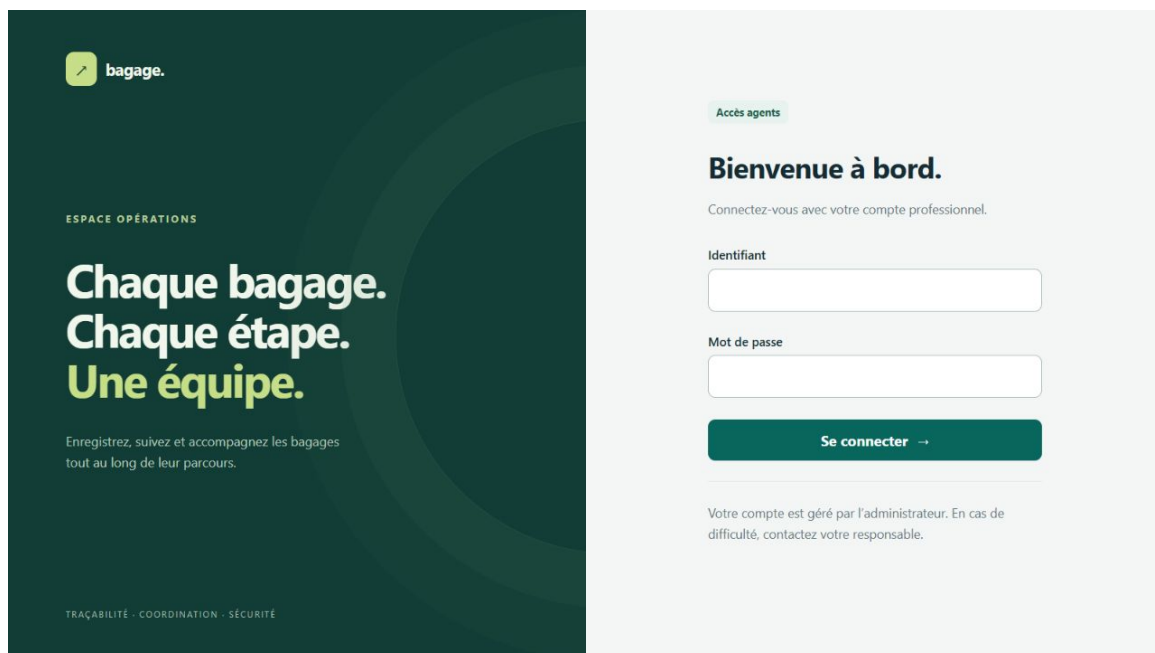


FIGURE 15 – Redirection vers la connexion agents

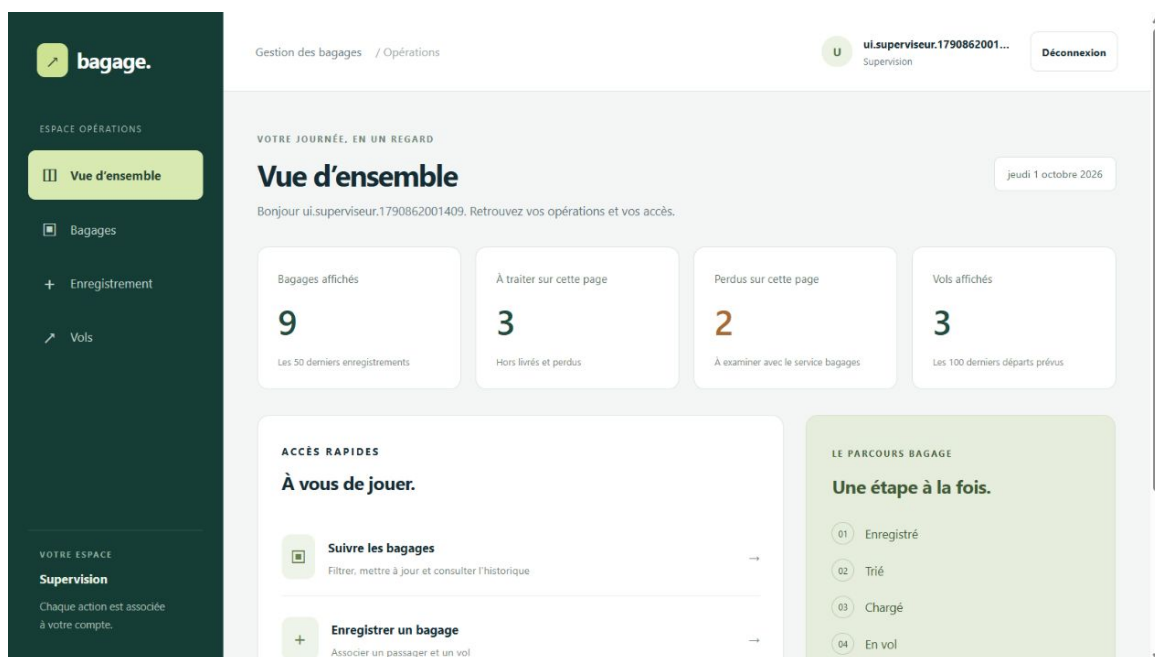


FIGURE 16 – Tableau de bord du superviseur

UI62281279 · SFA → ORY · 03/10 10:30

Nom du passager

Poids (kg)

L'identité du passager reste privée et n'apparaît pas dans le suivi public.

Enregistrer le bagage

Bagage enregistré Enregistré

Code de suivi à remettre au passager :

E636DBE90950104D8E4FE8435FDB6B9995A4CDAD79341CCD

Référence interne : e8cf98de-346f-484f-9279-f7c67c3a7a8f

22 4 kg · Conservez la référence interne pour consulter l'historique

FIGURE 17 – Enregistrement et reçu de suivi

Mettre à jour le parcours Fermer

E636DBE90950104D8E4FE8435FDB6B9995A4CDAD79341CCD

Statut actuel : Trié

Nouveau statut

Choisir une étape

Statut enregistré. L'historique a été mis à jour.

Enregistrer le statut

Historique du bagage Traçabilité

Enregistré

01/10/2026 à 14:46:29

Agent : f0657a27-27aa-459a-b0d3-296b8e815936

FIGURE 18 – Transition effectuée par l'agent de tri

3.4.4 Vérifier la continuité avec le suivi passager

Le code obtenu lors de l'enregistrement est saisi dans l'application publique. La consultation restitue l'état Trié et les horodatages du parcours. Les données viennent de la même API et de la même base que celles utilisées par les agents.

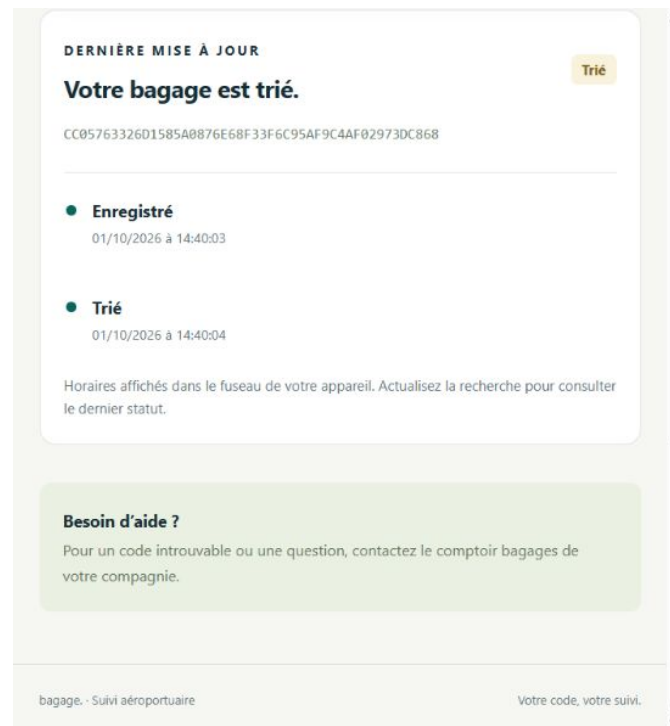


FIGURE 19 – Consultation du statut Trié

Cette étape ferme la boucle entre enregistrement, manutention et information du passager. La capture reflète l'état du bagage au moment de la consultation ; une correction ultérieure peut modifier ce statut sans invalider la trace historique de cette étape.

3.4.5 Conserver la trace des corrections exceptionnelles

Une transition qui sort du parcours normal demande l'intervention du superviseur et un motif. Le scénario de correction vers Livré vérifie que l'application conserve la justification et l'agent responsable, au lieu de masquer l'anomalie par un changement silencieux.

L'événement marqué comme exceptionnel permet de comprendre la correction. La traçabilité reste donc exploitable même lorsque le cycle normal n'a pas été suivi intégralement.

3.4.6 Isoler l'administration et contrôler l'origine de l'interface

Le compte administrateur est réservé à la gestion des utilisateurs. Une création de compte fictif est réalisée depuis son formulaire. L'application agents vérifie aussi son origine de service : une origine

The screenshot displays the 'bagage.' application interface. On the left is a dark green sidebar with the logo and navigation options: 'ESPACE OPÉRATIONS', 'Vue d'ensemble', 'Bagages' (highlighted), 'Enregistrement', and 'Vols'. Below this is 'VOTRE ESPACE' with 'Supervision' and a note: 'Chaque action est associée à votre compte.' The main content area shows a header with flight details: 'CC05763326D1...', 'UI2001409', 'Livré', '18.5 kg', '01/10 14:40', and an 'Ouvrir →' button. Below the header is a pagination bar: 'Page 1 - 1 résultat(s). 50 maximum par page' with 'Précédente' and 'Suivante' buttons. The main content is divided into two panels. The left panel, 'Mettre à jour le parcours', has a 'Fermer' button and shows the current status 'Statut actuel : Livré'. It includes a 'Nouveau statut' dropdown menu with 'Choisir une étape', a 'Motif de correction' text area, and a note: 'Obligatoire pour une transition hors parcours normal.' A green message box states: 'Statut enregistré. L'historique a été mis à jour.' and a button 'Enregistrer le statut' is at the bottom. The right panel, 'Historique du bagage', has a 'Traçabilité' button and shows a timeline of three events: 'Enregistré' (01/10/2026 à 14:40:03, Agent: f0657a27-27aa-459a-b0d3-296b8e815936), 'Enregistré → Trié' (01/10/2026 à 14:40:04, Agent: 2cc85372-8472-47b8-8ebb-eafa9623c104), and 'Trié → Livré' (01/10/2026 à 14:44:24, Agent: 5470f3b4-5ed3-4d15-bac2-70ed8951f29f). The last event is marked as 'Transition anormale' and includes a note: 'Démonstration : correction après confirmation de remise au passager fictif.'

FIGURE 20 – Correction exceptionnelle et historique

locale absente de la liste autorisée empêche le chargement du module.

The screenshot shows the 'Créer un compte' form. At the top, there's a header with 'ui-superviseur. / 750602001409' and 'supervision'. The form has three main input fields: 'Identifiant du compte' (a text box), 'Rôle' (a dropdown menu with 'Enregistrement' selected), and 'Mot de passe initial' (a text box). Below the password field is a note: '12 caractères minimum, avec majuscule, minuscule et chiffre.' and another note: 'Le compte Administrateur gère les utilisateurs. Les opérations sur les vols et bagages nécessitent un compte métier distinct.' At the bottom, a green message box states: 'Compte créé. Transmettez les identifiants par un canal privé.'

FIGURE 21 – Création d'un compte par l'administrateur

La confirmation de création ne révèle pas le mot de passe. Le refus d'origine constitue un contrôle

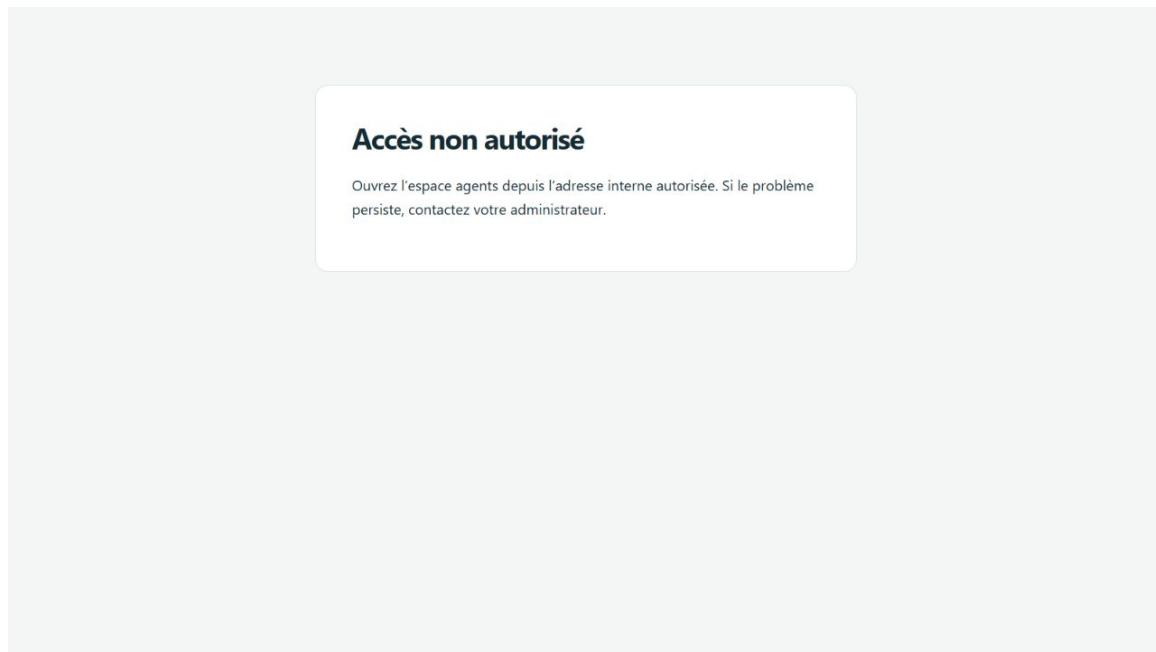


FIGURE 22 – Contrôle applicatif de l'origine

applicatif supplémentaire, distinct du VPN. Les parcours ayant été vérifiés, la réalisation aborde maintenant la protection réseau nécessaire aux opérations privées.

3.5 Sécurisation réseau : ajouter le VPN aux autorisations applicatives

3.5.1 Segmenter les conteneurs et limiter les publications

Les composants sont répartis sur des réseaux dédiés au point d'entrée, aux échanges avec l'API et à PostgreSQL. Nginx public relaie uniquement le suivi; le proxy agents reste sur le chemin privé. WireGuard ajoute la passerelle nécessaire aux clients habilités. La vue Docker Desktop permet de vérifier l'organisation obtenue.

La présence des services ne signifie pas que leurs ports sont accessibles à tous. Les fonctions privées ne sont plus publiées comme dans le laboratoire initial. Il reste à vérifier cette frontière depuis des clients placés avec et sans tunnel.

3.5.2 Tester le tunnel avec le même jeton

Un client autorisé établit le tunnel puis utilise un JWT valide pour atteindre le service privé. Le tunnel est coupé en conservant ce jeton, puis rétabli. Un second client, extérieur au VPN dans le laboratoire Docker, essaie les mêmes accès et consulte le suivi public.

La figure 24 isole l'effet du VPN en conservant le même JWT; la figure 25 complète cette preuve depuis un client extérieur au tunnel. L'accès privé disparaît à la coupure et revient avec le tunnel, tandis que le site public reste disponible. Ce résultat montre que le JWT ne suffit pas à franchir la

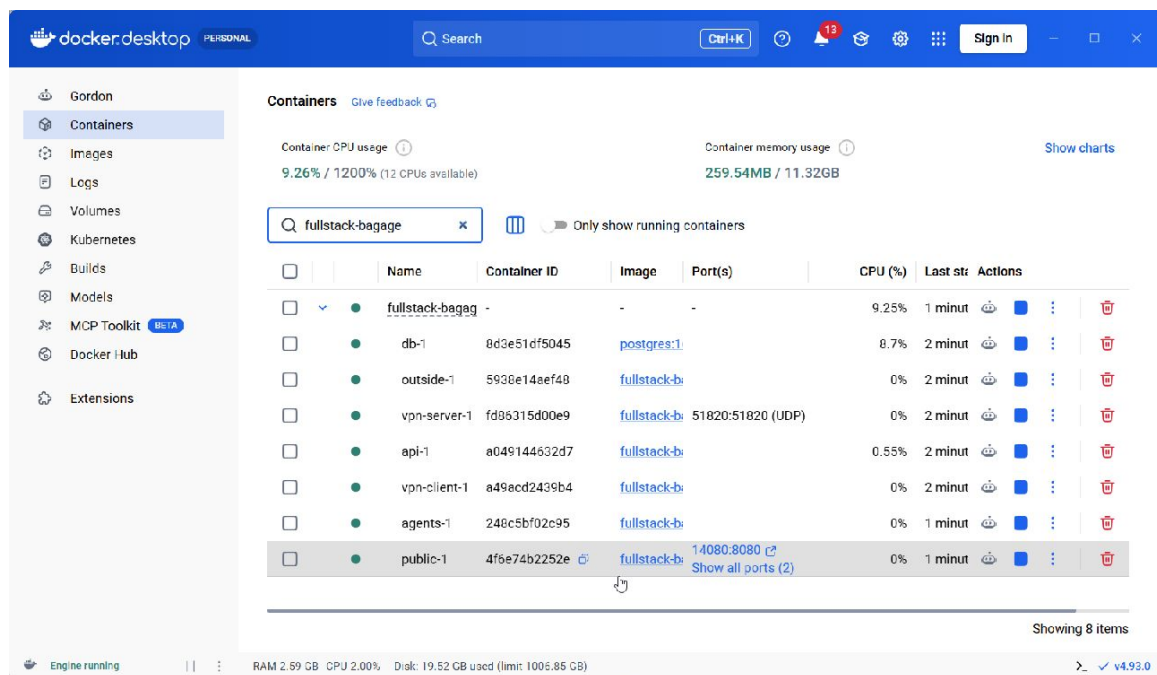


FIGURE 23 – Organisation des conteneurs réseau

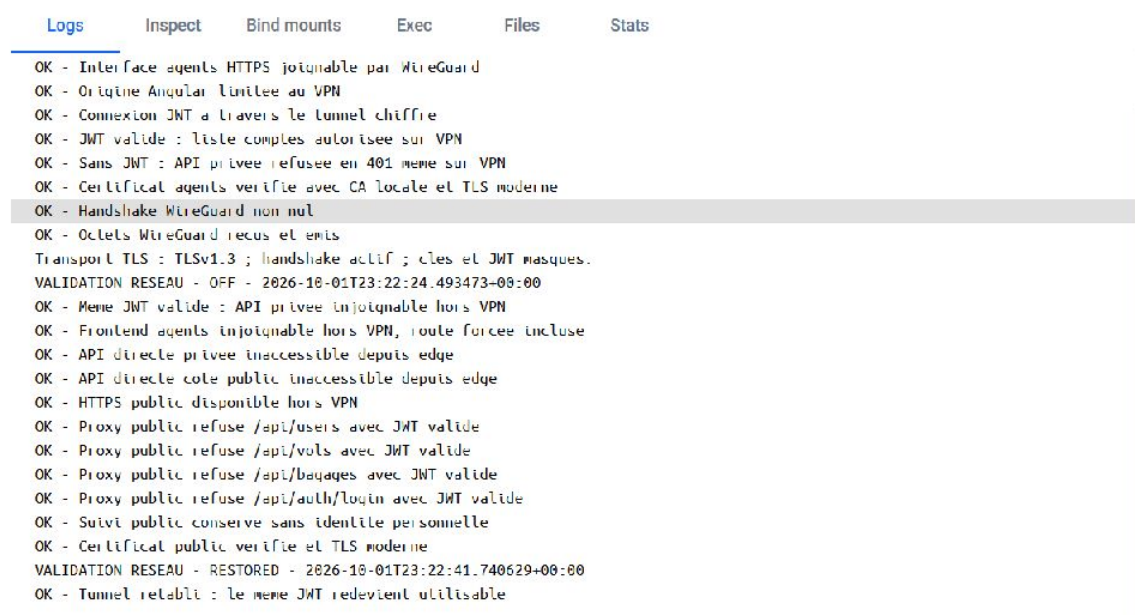


FIGURE 24 – Accès privé, coupure et rétablissement du tunnel

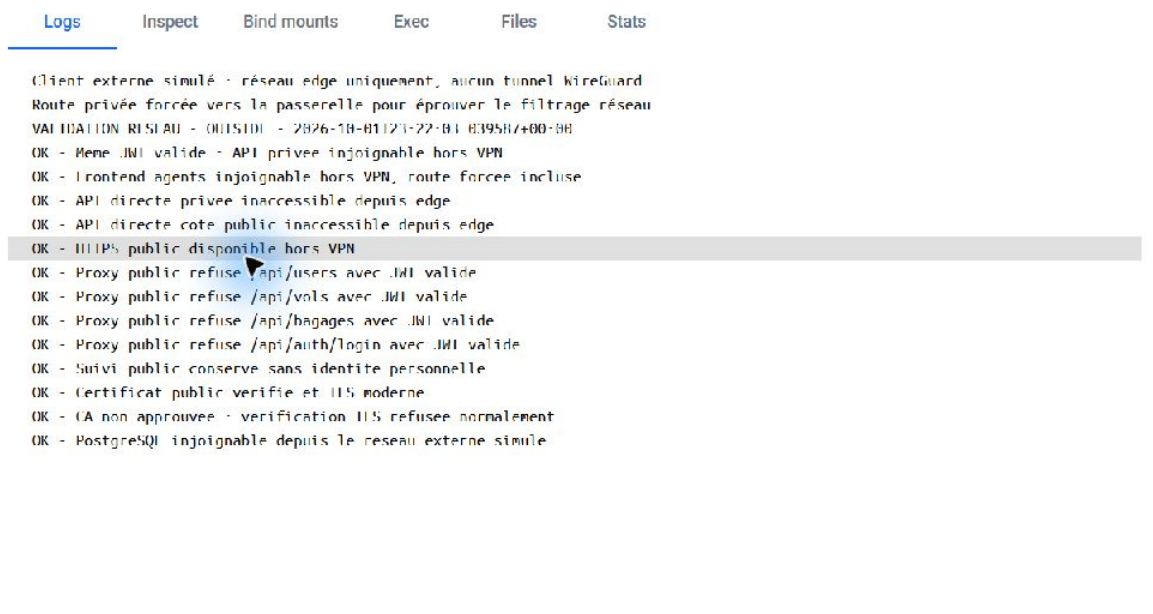


FIGURE 25 – Client extérieur au VPN dans le laboratoire

frontière réseau. À cette étape, les clients sont encore des conteneurs sur le même poste : la validation porte sur le laboratoire, avant son transfert vers Kubernetes.

3.6 Kubernetes : déployer les composants et vérifier la persistance

3.6.1 Installer le cluster et déclarer les ressources

Un cluster k3s mono-nœud est créé dans k3d. L'API et les interfaces utilisent des Deployments ; PostgreSQL utilise un StatefulSet et un volume persistant. Les migrations et l'initialisation passent par des Jobs. Les sorties kubectl enregistrées complètent la vue du nœud dans Docker Desktop.

Le lecteur de sorties kubectl visible dans Docker sert uniquement à consulter les preuves. Les services privés sont déclarés en ClusterIP, les flux sont limités par NetworkPolicy et le stockage reste local au nœud. Le déploiement doit ensuite être vérifié depuis ses points d'entrée.

3.6.2 Retrouver les protections réseau et conserver les données

Les essais avec et sans VPN sont reproduits contre Kubernetes. La validation contrôle également les politiques réseau et le remplacement du pod PostgreSQL. Après ce remplacement, le bagage et ses événements doivent rester disponibles grâce au volume persistant.

La frontière VPN est conservée après le changement de mode de déploiement. La persistance vérifiée concerne le remplacement du pod ; elle ne prouve pas une reprise après perte du disque. Une restauration de sauvegarde sera donc testée séparément.

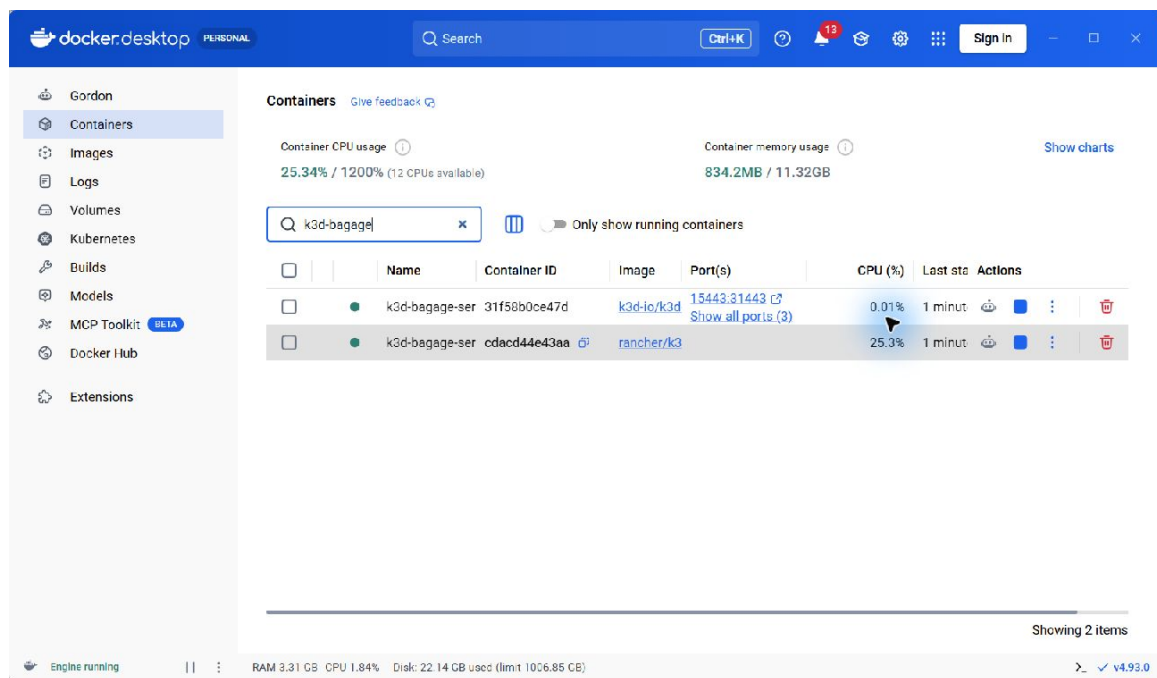


FIGURE 26 – Cluster k3s dans k3d

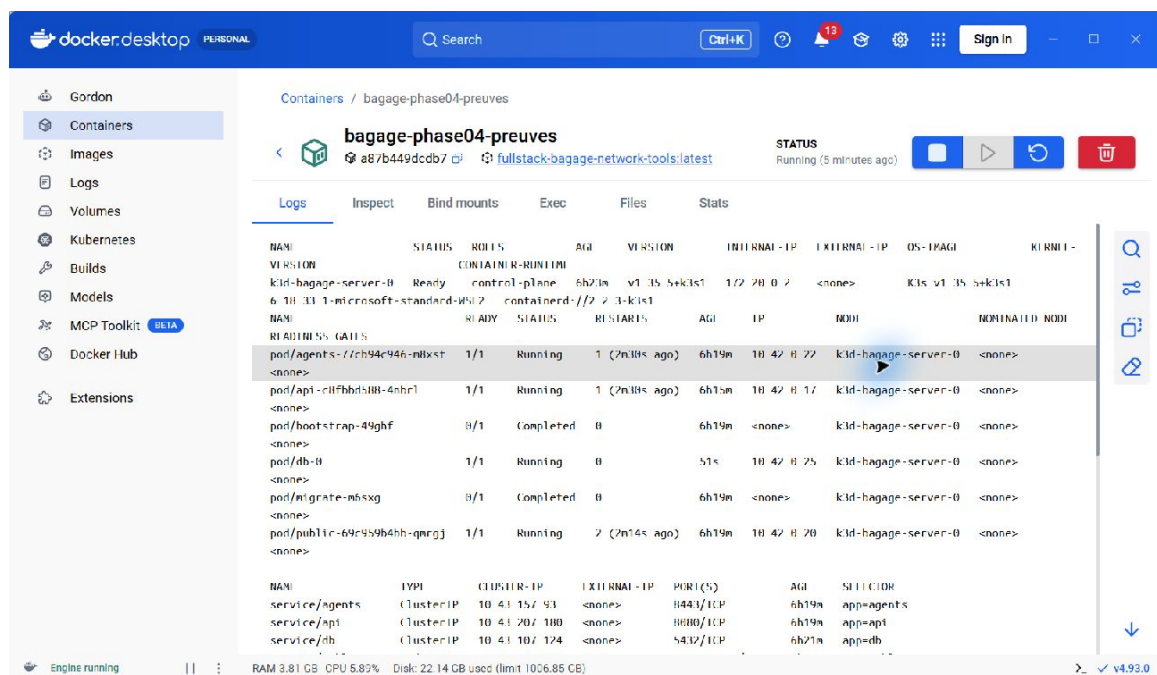


FIGURE 27 – Ressources observées avec kubectl

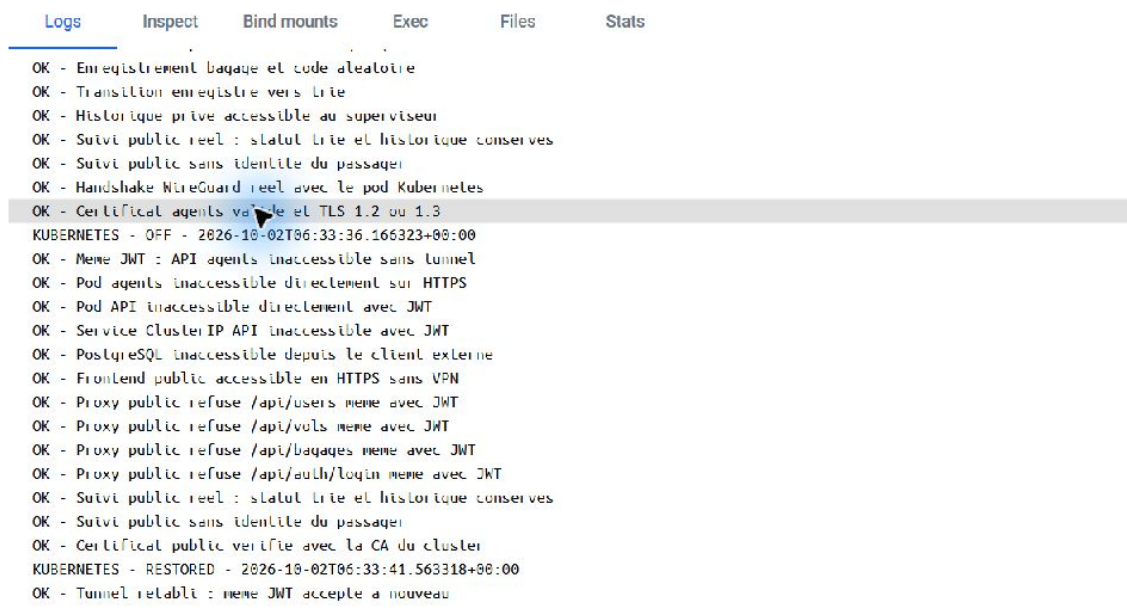


FIGURE 28 – WireGuard et services privés du cluster

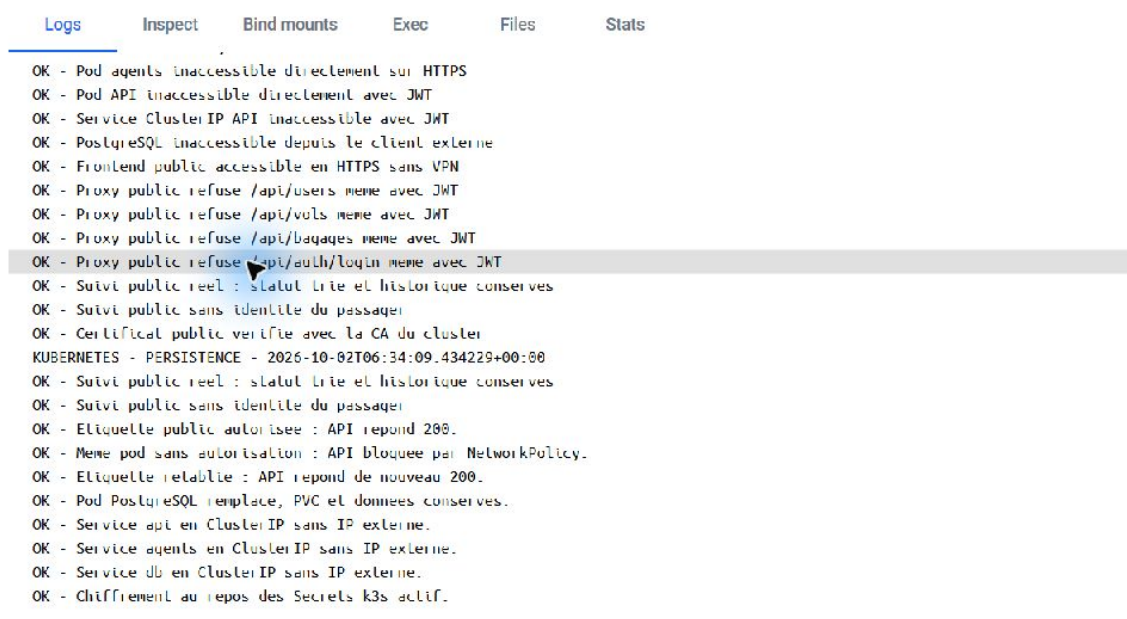


FIGURE 29 – Restrictions et persistance Kubernetes

3.6.3 Sortir du client Docker : Windows et Ubuntu VMware

Le client WireGuard natif de Windows est utilisé pour vérifier le chemin réel du poste vers le service privé. Une VM Ubuntu exécute ensuite la même famille de contrôles. Les résultats de ces systèmes sont archivés puis consultés dans Docker Desktop, qui joue ici le rôle de lecteur et non celui de client testé.

Logs	Inspect	Bind mounts	Exec	Files	Stats
networkpolicy.network.k8s.io/api-to-ap					app=api 6h21m
networkpolicy.network.k8s.io/bootstrap-to-db					app=bootstrap 6h21m
networkpolicy.network.k8s.io/database-from-api-and-migrations					app=db 6h21m
networkpolicy.network.k8s.io/deny-by-default					<none> 6h21m
networkpolicy.network.k8s.io/dns					<none> 6h21m
networkpolicy.network.k8s.io/migrate-to-db					app=migrate 6h21m
networkpolicy.network.k8s.io/public-https					app=public 6h21m
networkpolicy.network.k8s.io/public-to-api					app=public 6h21m
networkpolicy.network.k8s.io/vpn-udp					app=agents 6h21m
VALIDATION WINDOWS NATIVE - RESULTATS REELS					
OK - Interface agents accessible par VPN Windows					
OK - Authentification depuis Windows dans le tunnel					
OK - JWT valide accepte sur VPN Windows					
OK - Sans JWT : refus 401 meme sur VPN Windows					
OK - Handshake WireGuard Windows non nul ; clés masquées.					
OK - Meme JWT : API privée inaccessible VPN Windows coupe					
OK - Interface agents inaccessible VPN Windows coupe					
OK - Suivi public reste accessible sans VPN					
OK - Proxy public refuse users avec le meme JWT					
OK - Proxy public refuse vols avec le meme JWT					
OK - Proxy public refuse bagages avec le meme JWT					
OK - Proxy public refuse auth/login avec le meme JWT					
OK - Route privée WireGuard retirée de Windows après coupure.					
OK - VPN Windows rétabli : meme JWT accepte					

FIGURE 30 – Validation avec le client Windows natif

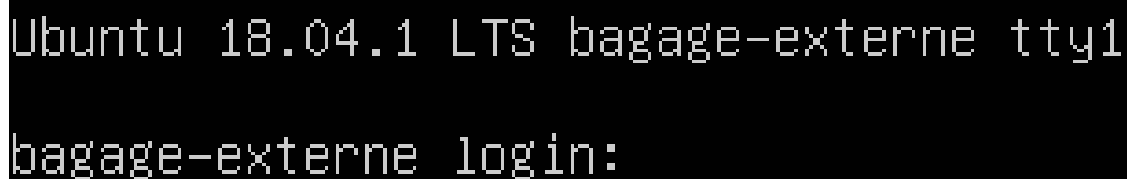
Logs	Inspect	Bind mounts	Exec	Files	Stats
OK - Certificat public valide et pas moderne					
OK - Sans CA explicite : certificat refuse					
OK - Baqage Kubernetes suivi depuis la VM					
OK - Aucune identité personnelle dans le suivi					
OK - Proxy public refuse /api/users					
OK - Proxy public refuse /api/vols					
OK - Proxy public refuse /api/bagages					
OK - Proxy public refuse /api/auth/login					
OK - Port privé hôte 8443 ferme depuis VM					
OK - Port privé hôte 18080 ferme depuis VM					
OK - Port privé hôte 14201 ferme depuis VM					
OK - HTTPS agents inaccessible sans VPN					
OK - VM : interface agents accessible avec TLS vérifiée					
OK - VM : authentification dans le tunnel					
OK - VM : JWT accepte sur API privée					
OK - VM : sans JWT accès privé refuse					
OK - Handshake WireGuard VM non nul ; clé masquée					
OK - VM : meme JWT inaccessible après coupure VPN					
OK - VM sans VPN : / avec meme JWT					
OK - VM sans VPN : /api/users avec meme JWT					
OK - VM sans VPN : /api/vols avec meme JWT					
OK - VM sans VPN : /api/bagages avec meme JWT					
OK - VM sans VPN : /api/auth/login avec meme JWT					
OK - VM : meme JWT accepte après rétablissement					

FIGURE 31 – Validation depuis Ubuntu VMware

Ces essais confirment le fonctionnement depuis Windows et un système invité Ubuntu. La VM reste sur le même poste physique : son utilisation ne suffit pas, à elle seule, à conclure à un accès VPN depuis Internet.

3.6.4 Préparer une VM dédiée aux validations

Une nouvelle VM est installée depuis l'ISO Ubuntu fourni, sur un disque distinct. Son premier démarrage confirme l'installation. Les tests réseau et VPN sont ensuite exécutés dans sa console, puis complétés par la consultation HTTPS du service public via une URL temporaire.



```
Ubuntu 18.04.1 LTS bagage-externe tty1
bagage-externe login:
```

FIGURE 32 – Premier démarrage de la VM Ubuntu



```
VM REELLE bagage-externe - Ubuntu 18.04.1 - Validation reseau
Adresse NAT : 192.168.233.137/24 | WireGuard : coupe apres les tests
-----
VALIDATION VM REELLE - bagage-externe - Linux
OK - Interface publique Angular joignable depuis la VM
OK - Certificat public verifie et TLS moderne
OK - Sans CA explicite : certificat refuse
OK - Bagage Kubernetes suivi depuis la VM
OK - Aucune identite personnelle dans le suivi
OK - Proxy public refuse /api/users
OK - Proxy public refuse /api/vols
OK - Proxy public refuse /api/bagages
OK - Proxy public refuse /api/auth/login
OK - Port prive hote 8443 ferme depuis VM
OK - Port prive hote 18080 ferme depuis VM
OK - Port prive hote 14201 ferme depuis VM
OK - HTTPS agents inaccessible sans VPN
OK - VM : interface agents accessible avec TLS verifie
OK - VM : authentication dans le tunnel
OK - VM : JWT accepte sur API privree
OK - VM : sans JWT acces prive refuse
OK - Handshake WireGuard VM non nul ; cle masquee
OK - VM : meme JWT inaccessible apres coupure VPN
OK - VM sans VPN : / avec meme JWT
OK - VM sans VPN : /api/users avec meme JWT
OK - VM sans VPN : /api/vols avec meme JWT
OK - VM sans VPN : /api/bagages avec meme JWT
OK - VM sans VPN : /api/auth/login avec meme JWT
OK - VM : meme JWT accepte apres retablissement
```

FIGURE 33 – Contrôles réseau et VPN dans la VM

La VM dédiée fournit un client de laboratoire reproductible. Les tests de tunnel local et les contrôles HTTPS publics sont distingués : seuls ces derniers empruntent l'URL Internet temporaire. La suite du

```
VM REELLE bagage-externe - Client NAT distinct du cluster
Resultats reels : HTTPS public via Internet, CA systeme verifiee
VM Ubuntu bagage-externe : HTTPS public via Internet, sans WireGuard
OK - Interface Angular Internet avec certificat systeme verifie
OK - Suivi reel Kubernetes via Internet
OK - Suivi Internet sans identite personnelle
OK - Internet : route privree refusee users
OK - Internet : route privree refusee vols
OK - Internet : route privree refusee bagages
OK - Internet : route privree refusee auth/login

Etat VPN actuel (aucun tunnel bagage-vm attendu) :

lo                UNKNOWN      127.0.0.1/8
enp0s17           UP            192.168.233.137/24
default via 192.168.233.2 dev enp0s17 proto dhcp src 192.168.233.137 metric 100
192.168.233.0/24 dev enp0s17 proto kernel scope link src 192.168.233.137
192.168.233.2 dev enp0s17 proto dhcp scope link src 192.168.233.137 metric 100
```

FIGURE 34 – HTTPS public depuis la VM dédiée

travail examine séparément la fermeture des ports locaux et le tunnel Internet.

3.6.5 Vérifier les ports du poste partagé

Les services Windows étrangers au projet sont liés à localhost afin de ne plus les exposer sur le LAN. Depuis la seconde VM, les ports concernés sont contrôlés, puis les accès par VPN local et les restrictions du site public sont vérifiés.

Les contrôles présentés valident le LAN et le VPN local. Leur portée reste distincte d'un handshake WireGuard par Internet, qui demande un point d'entrée accessible depuis un réseau extérieur.

```
bagage-scan
SECONDE VM - PORTS PARTAGES FERMES + VPN PAR CHEMIN LOCAL
Les tests Internet UDP ne sont PAS valides.

OK - 192.168.1.123:80 inaccessible depuis VM
OK - 192.168.1.123:5432 inaccessible depuis VM
OK - 192.168.1.123:8080 inaccessible depuis VM
OK - 192.168.1.123:5005 inaccessible depuis VM
OK - VM : interface agents accessible avec TLS verifie
OK - VM : authentication dans le tunnel
OK - VM : JWT accepte sur API privee
OK - VM : sans JWT acces prive refuse
OK - Handshake WireGuard VM non nul ; cle masquee
OK - VM : meme JWT inaccessible apres coupure VPN
OK - VM sans VPN : / avec meme JWT
OK - VM sans VPN : /api/users avec meme JWT
OK - VM sans VPN : /api/vols avec meme JWT
OK - VM sans VPN : /api/bagages avec meme JWT
OK - VM sans VPN : /api/auth/login avec meme JWT
OK - VM : meme JWT accepte apres retablissement

Le serveur fonctionne localement ; aucun UDP mobile recu.
```

FIGURE 35 – Ports LAN fermés et VPN local

3.7 Recette et premier accès Internet

3.7.1 Consulter le suivi par HTTPS et vérifier la restauration

Un relais HTTPS temporaire permet de consulter le suivi public depuis Internet. Le bagage fictif apparaît avec ses événements. En parallèle, un dump PostgreSQL est restauré dans une base temporaire indépendante; les cinq tables, leurs empreintes et les droits de l'historique sont comparés.

La consultation confirme la continuité du suivi au travers du relais. La restauration logique vérifie l'utilisabilité de la sauvegarde sans remplacer la base active. L'URL temporaire est arrêtée après les essais; elle ne constitue pas l'hébergement final.

3.7.2 Valider WireGuard au travers d'un relais UDP public

Un relais UDP temporaire fournit ensuite un chemin Internet vers WireGuard. La VM réalise le handshake, utilise le JWT, coupe le tunnel et le rétablit. Les contrôles complémentaires vérifient le refus d'un pair inconnu et les restrictions des routes publiques.

Ces observations constituent une première validation Internet du tunnel, dans le périmètre du relais. Les services partagés du fournisseur ne sont pas scannés et le relais est arrêté après utilisation. Pour la soutenance, ce mécanisme temporaire est remplacé par un VPS disposant de son propre point d'entrée.

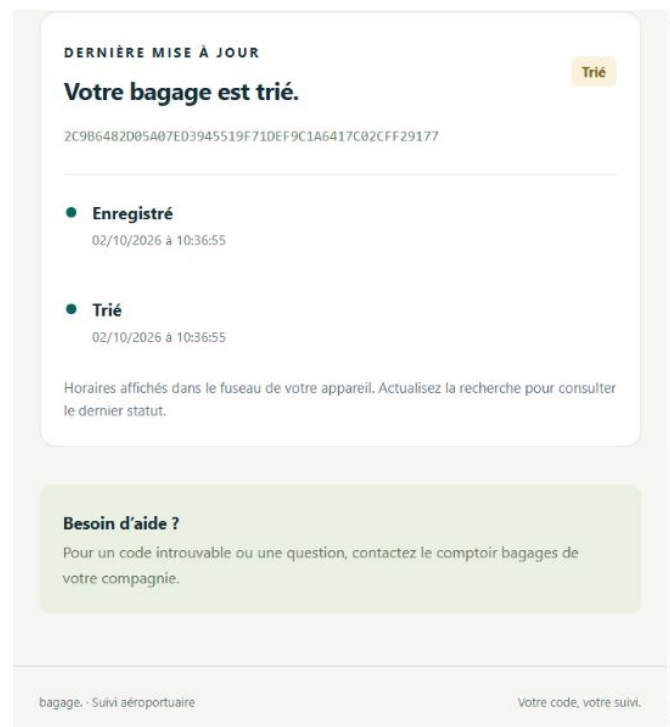


FIGURE 36 – Suivi public par URL Internet temporaire

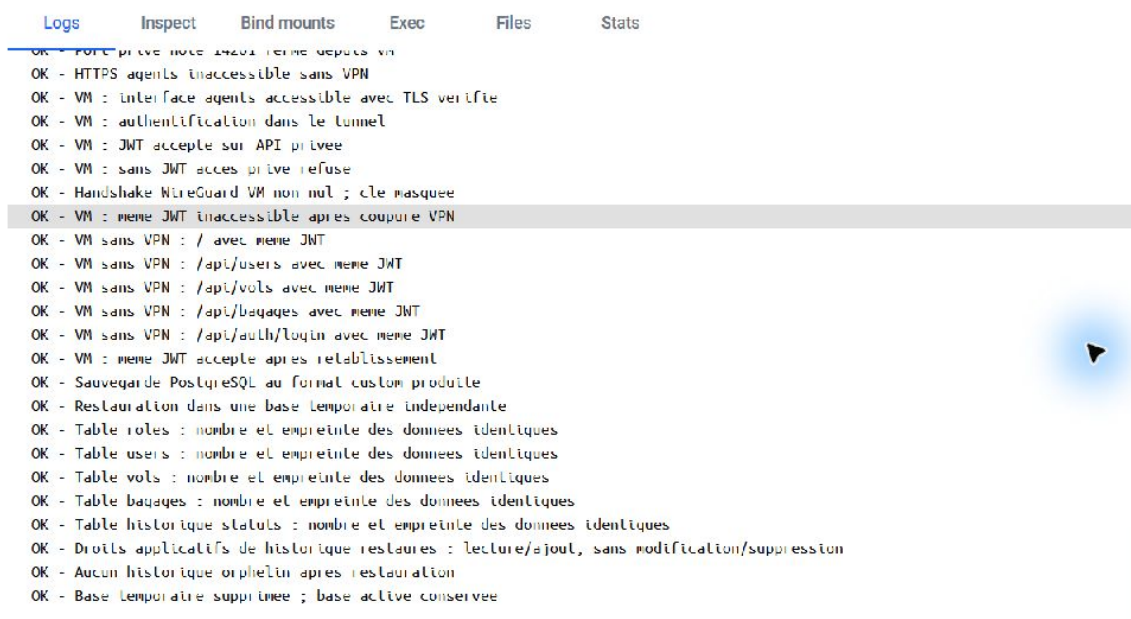


FIGURE 37 – Restauration logique vérifiée

```
bagage-scan
WIREGUARD VIA INTERNET - RELAIS UDP PUBLIC PINGGY
Resultats executes dans la VM ; aucune cle ni JWT affiche

OK - VM : interface agents accessible avec TLS verifie
OK - VM : authentication dans le tunnel
OK - VM : JWT accepte sur API privee
OK - VM : sans JWT acces prive refuse
OK - Handshake WireGuard VM non nul ; cle masquee
OK - VM : meme JWT inaccessible apres coupure VPN
OK - VM sans VPN : / avec meme JWT
OK - VM sans VPN : /api/users avec meme JWT
OK - VM sans VPN : /api/vols avec meme JWT
OK - VM sans VPN : /api/bagages avec meme JWT
OK - VM sans VPN : /api/auth/login avec meme JWT
OK - VM : meme JWT accepte apres retablissement

Le point entree WireGuard est un endpoint UDP public du relais.
Le trajet direct par redirection du routeur reste non valide.
```

FIGURE 38 – WireGuard Internet avec relais public

```
bagage-scan
CONTROLES INTERNET - SEULS POINTS ENTREE ALLOUES AU PROJET
UDP WireGuard public + URL HTTPS publique

OK - Pair WireGuard inconnu : aucun handshake Internet
OK - Pair inconnu : HTTPS prive inaccessible
OK - VPN Internet valide : faux JWT refuse
OK - Endpoint WireGuard : adresse publique du relais alloue
OK - Pair autorise : handshake Internet recent
OK - WireGuard Internet : octets recus et envoyes
OK - Scan HTTPS public : refus /api/users
OK - Scan HTTPS public : refus /api/vols
OK - Scan HTTPS public : refus /api/bagages
OK - Scan HTTPS public : refus /api/auth/login
OK - Scan HTTPS public : Swagger API non publie

Swagger renvoie le fallback Angular public, pas Swagger API.
Aucun scan des autres ports partages du fournisseur.
Ports PC 80/5432/8080/5005 limites a localhost (preuve separee).
```

FIGURE 39 – Restrictions et refus d'un pair inconnu

3.8 Hébergement OVH : rendre la démonstration indépendante du poste

3.8.1 Publier le service passager avec un certificat reconnu

Le déploiement est transféré vers le VPS Ubuntu 24.04. Le point d'entrée HTTPS public utilise un certificat Let's Encrypt reconnu et sert l'application Angular. Le parcours de suivi est repris avec un bagage fictif pour vérifier que les réponses proviennent désormais du serveur distant.

The screenshot displays the 'bagage.' website interface. At the top, there is a logo 'bagage.' and the tagline 'LE SUIVI, TOUT SIMPLEMENT'. A button labeled 'Espace passagers' is in the top right corner. The main heading reads 'DU DÉPART À L'ARRIVÉE' followed by 'Votre voyage. Votre bagage. L'esprit tranquille.' Below this, it says 'Retrouvez les étapes de votre bagage, de son enregistrement à sa livraison.' and '✦ Sans compte. Sans donnée personnelle affichée.'

On the right, a white box titled 'SUIVI DES BAGAGES' contains the question 'Où en est mon bagage ?' and the instruction 'Saisissez le code qui vous a été remis lors de l'enregistrement.' It features a text input field for the tracking code, a note 'Le code figure sur votre reçu d'enregistrement.', and a green button 'Suivre mon bagage' with a right arrow. A checkbox at the bottom says 'Conservez votre code de suivi à l'abri des regards.'

Below the main content, a section titled 'CHAQUE ÉTAPE COMPTE' shows 'Un parcours, cinq étapes.' with a progress bar: 01 Enregistré, 02 Trié, 03 Chargé, 04 En vol, and 05 Livré. A green box at the bottom left offers 'Besoin d'aide ?' with contact information for the baggage counter.

The footer includes 'bagage. - Suivi aéroportuaire' on the left and 'Votre code, votre suivi.' on the right.

FIGURE 40 – Application publique hébergée sur OVH

Le site public est disponible tant que le VPS et son abonnement restent actifs, même lorsque le poste local est éteint. Le statut Trié et les deux événements sont conservés sur OVH ; aucune identité du passager n'est affichée.

3.8.2 Contrôler la frontière privée depuis Internet

WireGuard écoute directement sur UDP 52820 du VPS. La VM vérifie l'inaccessibilité des fonctions privées sans VPN et les huit ports TCP sensibles documentés. Sur Windows, le tunnel autorisé permet ensuite de charger l'interface agents en HTTPS avec son certificat privé reconnu.

bagage. LE SUIVI, TOUT SIMPLEMENT Espace passagers

— DU DÉPART À L'ARRIVÉE

Votre voyage. Votre bagage. L'esprit tranquille.

Retrouvez les étapes de votre bagage,
de son enregistrement à sa livraison.

• Sans compte. Sans donnée personnelle affichée.

SUIVI DES BAGAGES

Où en est mon bagage ?

Saisissez le code qui vous a été remis lors de l'enregistrement.

Code de suivi

AD4EA70B2694671525038121850507A44477E8608C89E7

Le code figure sur votre reçu d'enregistrement.

Suivre mon bagage

• Conservez votre code de suivi à l'abri des regards.

DERNIÈRE MISE À JOUR

Votre bagage est trié.

AD4EA70B2694671525038121850507A44477E8608C89E7

Enregistré 02/10/2026 à 19:25:02

Trié 02/10/2026 à 19:25:02

Horaires affichés dans le foyou de votre appareil. Actualisez la recherche pour consulter le dernier statut.

Besoin d'aide ?

Pour un code introuvable ou une question, contactez le comptoir bagages de votre compagnie.

bagage. - Suivi aéroportuaire

Votre code, votre suivi.

FIGURE 41 – Suivi du bagage fictif sur OVH

```

OK - Site public HTTPS avec certificat reconnu
OK - API privée inaccessible hors VPN avec le même JWT
OK - Route publique privée refusée users
OK - Route publique privée refusée bagages
OK - Route publique privée refusée vols
OK - Route publique privée refusée auth/login
OK - Port TCP OVH fermé 5432
OK - Port TCP OVH fermé 8080
OK - Port TCP OVH fermé 8443
OK - Port TCP OVH fermé 5005
OK - Port TCP OVH fermé 6443
OK - Port TCP OVH fermé 16443
OK - Port TCP OVH fermé 31443
OK - Port TCP OVH fermé 31820
OK - Swagger non publiée

```

FIGURE 42 – Scan Internet du VPS sans VPN

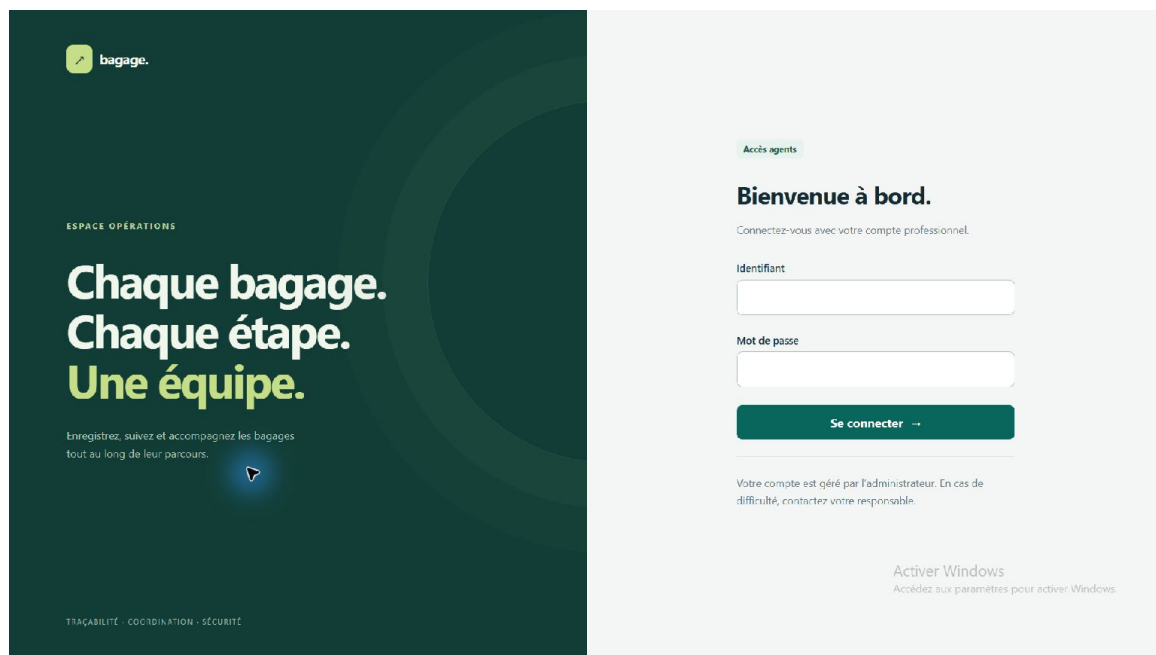


FIGURE 43 – Interface agents atteinte par WireGuard OVH

La page de connexion montre le chargement réel de l'interface privée, sans prétendre représenter une session métier déjà ouverte. Les scénarios API complètent cette observation : refus sans jeton, refus d'un pair inconnu, coupure et rétablissement avec le même JWT. Les vérifications IPv4 et IPv6 portent sur les routes et ports documentés, sans revendiquer un scan exhaustif.

3.8.3 Assurer la maintenance et vérifier la reprise

L'administration SSH utilise une clé ; les mots de passe SSH sont désactivés après validation de cet accès. Le renouvellement du certificat public est simulé et son hook met à jour le secret TLS. PostgreSQL [6] fournit `pg_dump` pour les sauvegardes logiques : un dump quotidien est planifié à 02:15 UTC et les sept dernières sauvegardes réussies sont conservées.

Une copie manuelle hors VPS possède la même empreinte que le dump serveur. La restauration est vérifiée dans une base distincte. Enfin, un redémarrage réel du VPS remet les quatre pods applicatifs en état prêt et conserve les trois bagages de démonstration. Ces résultats vérifient la reprise après redémarrage, pas la haute disponibilité ni la perte complète du nœud.

3.9 Bilan de validation et correspondance avec le cahier des charges

La validation reprend les critères du tableau 1 : chaque propriété est reliée à une condition de réussite et à une preuve. Elle associe appels HTTP, parcours métier, accès réseau et observations dans les interfaces. Les rejeux ne sont pas comptés deux fois. Le bilan OVH rassemble les contrôles suivants, exécutés après le déploiement distant.

TABLE 8 – Contrôles OVH réussis enregistrés le 2 octobre 2026

Groupe	Nombre
Recette métier	49
Internet depuis Docker	39
Internet depuis VMware	32
Infrastructure et durcissement	26
Restauration	10
Windows natif	5
IPv6	13
Total OVH	174

Les 281 contrôles et observations antérieurs portent le bilan à 455. Ce nombre ne mesure pas la couverture des tests et ne constitue pas une certification de sécurité.

TABLE 9 – Traçabilité des exigences et de leur validation

Exigence	Réalisation et preuve	Périmètre
Suivi public sans identité	Formulaire Angular, route de suivi dédiée; captures phases 1, 2 et 5.	Local et OVH
Enregistrement et cycle métier	Parcours des rôles, transitions normales, perte et correction motivée; recette métier.	Application
Authentification et rôles	JWT vérifié, refus sans jeton et avec rôle insuffisant; comptes séparés.	API
Traçabilité obligatoire	Événements atomiques, horodatages, agent et motif; historique et recette.	Base et API
Accès privé par VPN	Coupure/rétablissement avec le même JWT, pair inconnu refusé; phases 3 à 5.	Réseau
Absence d'exposition PostgreSQL	Service privé et scans des ports connus depuis les clients de test.	Ports testés
Déploiement Kubernetes	Deployments, StatefulSet, PVC, Jobs et NetworkPolicy; sorties kubectl.	Un seul nœud
Sauvegarde et restauration	Dump quotidien sur OVH, restauration logique séparée et copie manuelle hors VPS.	Démonstration
Rapport et preuves	Guides, résultats expurgés, 37 captures présentées et leurs empreintes.	Livrables

3.10 Analyse des résultats et portée des tests

Les résultats sont interprétés en fonction de la propriété réellement vérifiée. Le refus attendu d'une opération interdite est un succès du contrôle de sécurité. Les tests fonctionnels et les tests réseau répondent à des questions différentes et se complètent.

TABLE 10 – Interprétation des validations et limites de leurs conclusions

Expérience	Conclusion étayée	Limite
Sans JWT ou rôle insuffisant	L'API refuse les opérations non autorisées dans les scénarios testés.	Ne démontre pas à elle seule la fermeture d'un service réseau.
Même JWT avec coupure VPN	L'accès privé dépend du tunnel, en plus du jeton valide.	Porte sur les routes et clients testés.
Suivi public et routes privées	Le passager obtient un suivi limité; le proxy ne relaie pas les fonctions internes testées.	Le code de suivi doit rester confidentiel.
Pair WireGuard inconnu	Le client non configuré ne crée pas le chemin privé attendu.	Ne constitue pas un audit cryptographique du protocole.
Ports sensibles IPv4/IPv6	Les sondes extérieures ne rejoignent pas les ports privés documentés.	Scan limité, sans couverture de tous les ports.
Remplacement du pod DB	Les données survivent à la recreation du pod grâce au volume.	Pas de preuve de reprise après perte du disque.
Dump et restauration	La copie logique restaure les tables, leurs données et les droits contrôlés.	Restauration sur le même serveur; copie hors VPS manuelle.
Redémarrage OVH	Les services redémarrent et les bagages contrôlés sont conservés.	Pas de haute disponibilité ni test de charge.

Le nombre de contrôles traduit l'étendue des expériences consignées, et non un pourcentage de couverture du code. Les captures complètent les résultats textuels : elles rendent visible l'usage ou l'exécution, sans remplacer le détail des assertions. La conclusion retenue est celle d'un démonstrateur fonctionnel et sécurisé dans le périmètre présenté.

3.11 Apports du travail et compétences mobilisées

Le travail réalisé associe trois contributions : la traduction des responsabilités métier en opérations contrôlées, la construction d'une frontière réseau distincte de l'authentification, et la préparation d'un déploiement accompagné de preuves reproductibles. La valeur de la réalisation réside dans

leur articulation : le même parcours est conservé lorsque l'environnement évolue du poste local vers Docker, Kubernetes puis OVH.

3.11.1 Décisions de conception et enseignements

La séparation des applications et des proxys évite de confondre un menu masqué avec une route protégée. Le compte administrateur dédié limite également le mélange entre gestion des utilisateurs et opérations sur les bagages. Ces décisions traduisent le moindre privilège en comportements que la recette peut observer.

La distinction entre un client Docker, une VM et un client passant réellement par Internet constitue un autre enseignement. Une VM apporte un environnement différent, mais sa présence sur le même poste ne prouve pas à elle seule un trajet extérieur. Le VPS et les essais depuis les clients documentés permettent de préciser le périmètre réel des résultats, sans généraliser une preuve locale.

Le passage à Kubernetes impose enfin de séparer le cycle de vie du conteneur de celui des données. Le volume protège les données lors du remplacement du pod ; la sauvegarde répond à un autre besoin. La restauration dans une base distincte apporte une preuve supplémentaire, sans démontrer une reprise après perte totale de l'hôte.

3.11.2 Livrables et autonomie de reproduction

Les sources, migrations, manifestes, scripts et guides constituent les livrables techniques. Les commandes documentées et les résultats associés permettent de reprendre une étape, de localiser la protection testée et de préparer la démonstration. Les captures illustrent les observations ; les assertions conservées dans les comptes rendus apportent les critères de validation. Cette distinction renforce la qualité du dossier au-delà de la seule présentation des interfaces.

Le travail mobilise la modélisation des données, le développement d'une API et d'interfaces, l'administration Linux, la conteneurisation, la segmentation et l'analyse des flux réseau. Il montre aussi l'importance de distinguer une réussite fonctionnelle d'une preuve de sécurité, et une persistance locale d'une stratégie complète de reprise.

3.12 Limites et perspectives

La réalisation constitue un démonstrateur mono-nœud. La copie hors VPS reste manuelle, les alertes continues ne sont pas validées et la perte complète du disque n'a pas été simulée. Une exploitation réelle demanderait une stratégie de haute disponibilité, de sauvegarde hors site automatisée, de supervision et de gestion des mises à jour.

3.13 Conclusion du chapitre

La progression part d'une chaîne API-base fonctionnelle, la rend utilisable par les différents rôles, puis ajoute les frontières réseau et la persistance. Les clients Windows et VMware élargissent la validation; le VPS termine la démarche en fournissant un hébergement Internet direct. Chaque série de captures intervient ainsi dans l'explication du résultat qui permet de passer à l'étape suivante.

Conclusion générale

Ce projet répond à la problématique de suivi des bagages en associant une application métier à une séparation explicite des accès publics et privés. Le passager consulte un code de suivi sans identité exposée; les agents interviennent selon leur rôle, au travers du réseau autorisé. L'historique conserve les changements d'état et les corrections motivées.

La démarche a progressé du modèle de données et de l'API vers les interfaces Angular, puis vers WireGuard, Kubernetes et l'hébergement OVH. Les validations montrent le fonctionnement des parcours, le refus des accès interdits, la persistance après remplacement d'un pod et la reprise après redémarrage. La restauration logique complète ces observations. Les résultats restent associés aux clients, routes et expériences documentés.

L'apport du projet est de relier les autorisations applicatives à une frontière réseau vérifiable et de fournir des configurations, scripts et preuves reproductibles. Cette réalisation développe des compétences complémentaires en développement, administration système et sécurité des infrastructures, en cohérence avec la spécialité ARSI.

Les perspectives concernent la haute disponibilité, la sauvegarde hors site automatisée, les alertes de supervision, les tests de charge et une revue de sécurité indépendante. Leur mise en œuvre nécessiterait de nouveaux essais et des ressources adaptées. Le résultat actuel constitue une base de démonstration pour la soutenance; il n'est pas présenté comme une infrastructure aéroportuaire de production.

Références

- [1] M. Jones, J. Bradley et N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, mai 2015. <https://www.rfc-editor.org/rfc/rfc7519>.
- [2] WireGuard, *Conceptual Overview*, documentation officielle. <https://www.wireguard.com/>.
- [3] Docker, *What is a container?*, documentation officielle. <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>.
- [4] Kubernetes, *Network Policies*, documentation officielle. <https://kubernetes.io/docs/concepts/services-networking/network-policies/>.
- [5] Angular, *What is Angular?*, documentation officielle. <https://angular.dev/overview>.
- [6] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, notamment le chapitre 26 sur la sauvegarde et la restauration. <https://www.postgresql.org/docs/16/index.html>.